

The University of Faisalabad

NAME : **LAIBA FATIMA**

CLASS : **BS-AI 6th Semester**

SECTION : **A**

COURSE TITLE : **DATA MINING**

COURSE CODE : **DS-303**

INSTRUCTOR : **Mr. ARHAM**

The University of Faisalabad
Department of Computational Sciences

Contents

LAB-1.....	3
LAB-2.....	17
LAB-3.....	26
LAB-4.....	31
LAB-5.....	35
LAB-6.....	40
LAB-7.....	43
LAB-8.....	46
LAB-9.....	49
LAB-10.....	52
LAB-11.....	55
LAB-12.....	58
LAB-13.....	61
LAB-14.....	63
Lab 15:	69

LAB-1

TOPICS :

1. Introduction to Data Mining
2. Software Introduction
3. Overview of datasets

CLO:

Understand data mining concepts, preprocessing, statistics, and trends. By doing this course we will be able to work on real industry level projects in future and our basics and foundation of studies would be good .

OBJECTIVE:

1. Understand the concept and applications of Data Mining
2. Use Google Colab as a development environment
3. Identify different types of datasets used in Data Mining

REQUIRED SOFTWARE / TOOLS

- Web Browser
- Google Account
- Google Colab
- Python (pre-installed in Colab)

INTRODUCTION TO DATA MINING

Data Mining is the process of discovering **useful patterns, trends, and knowledge** from large datasets using statistical, machine learning, and computational techniques.

Applications of Data Mining:

- Market basket analysis
- Fraud detection
- Healthcare diagnosis
- Social media analysis
- Recommendation systems

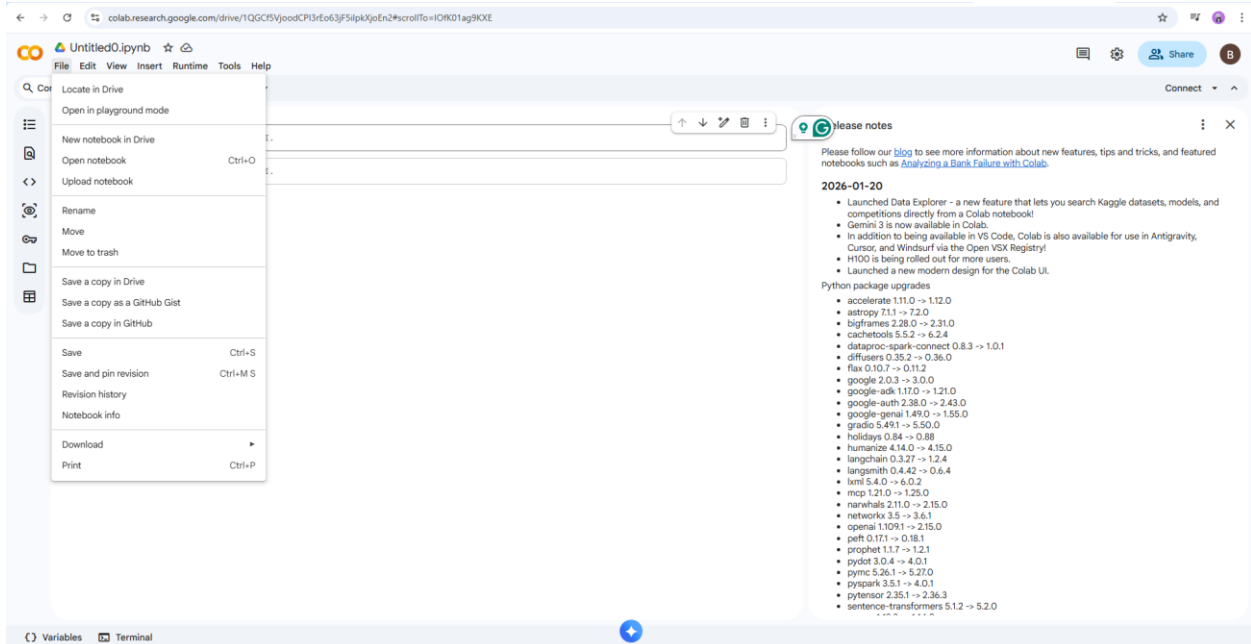
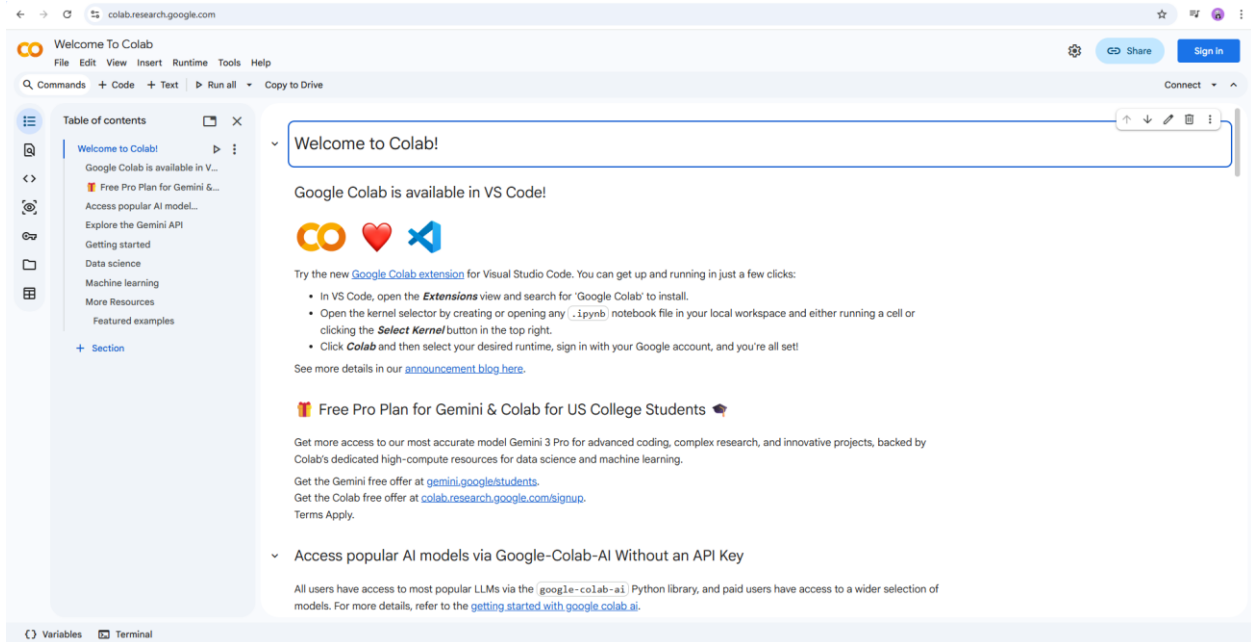
SOFTWARE INTRODUCTION – GOOGLE COLAB

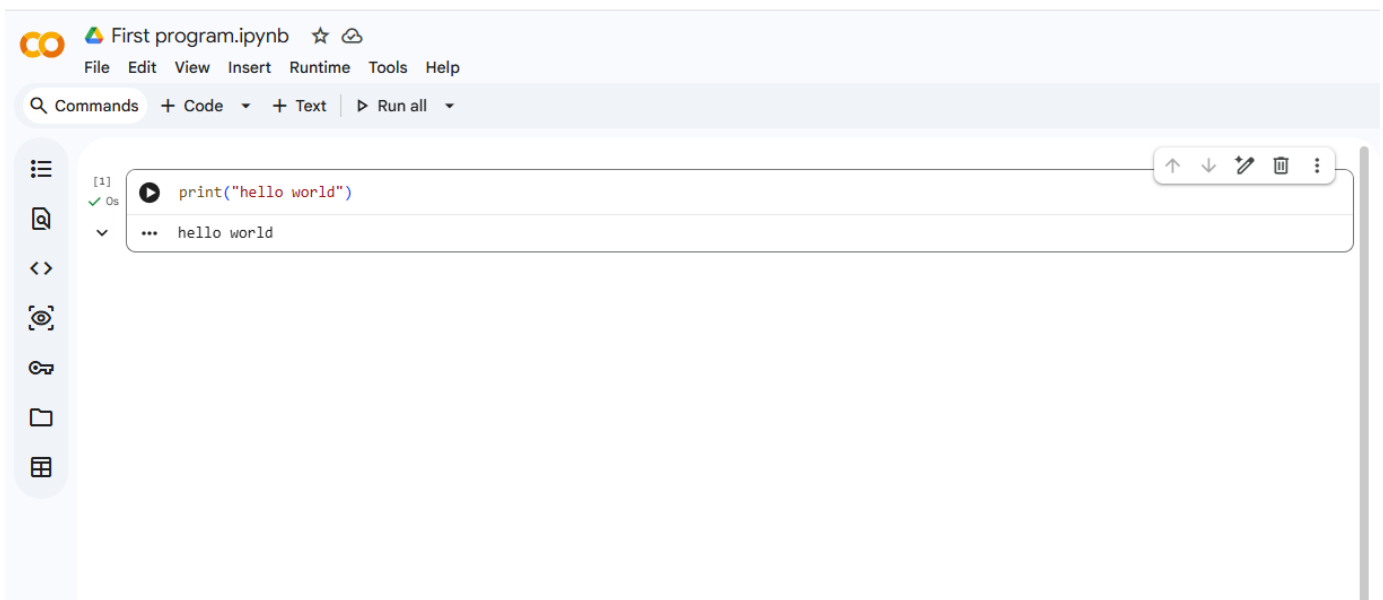
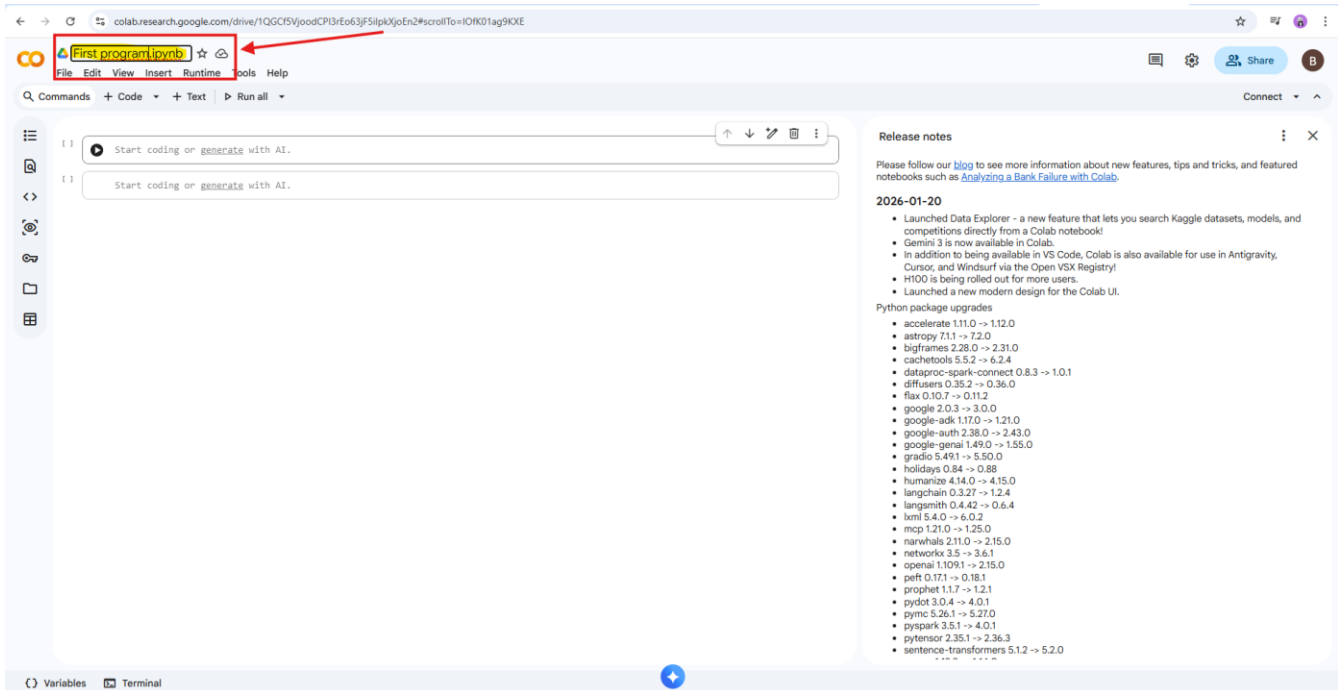
What is Google Colab?

Google Colaboratory (Colab) is a free cloud-based platform that allows users to write and execute Python code using a browser.

Features

- No installation required
- Free access to GPU/TPU
- Easy sharing and collaboration
- Pre-installed libraries (NumPy, Pandas, Matplotlib, etc.)





Overview of Datasets

What is data mining

A **dataset** is a structured collection of data used to discover patterns, relationships, or knowledge through data mining techniques.

In labs, datasets are usually stored as **tables** (rows = records, columns = attributes).

COMMON TYPES OF DATASETS

Structured Datasets

Data is organized in rows and columns.

Examples:

- Student performance data
- Sales transactions
- Bank customer records

Formats: CSV, Excel, SQL tables

Used for: Classification, clustering, regression

Transactional Datasets

Each record represents a transaction with a set of items.

Example:

```
T1 → {Milk, Bread}
T2 → {Milk, Diaper, Beer}
```

Used for:

- Association rule mining
 - Market basket analysis
- (Algorithms: Apriori, FP-Growth)

Time-Series Datasets

Data collected over time at regular intervals.

Examples:

- Stock prices
- Temperature readings
- Website traffic per day

Used for:

- Trend analysis
- Forecasting
- Sequential pattern mining

Spatial or Geographical Datasets

Data related to location or space.

This kind of data is very useful for satellites

Examples:

- GPS coordinates
- Maps
- Satellite images

Used for:

- Spatial clustering
- Location-based analysis

Text Datasets

Data in textual form.

Examples:

- Emails
- Reviews
- Tweets

Used for:

- Text mining
- Sentiment analysis
- Topic modeling

Web and Log Datasets

Data generated from web activity.

Examples:

- Clickstream data
- Server logs
- User browsing history

Used for:

- Web usage mining
- Recommendation systems

Attributes in a Dataset

1 Numerical Attributes

- Integer or real values
- Example: Age, Salary

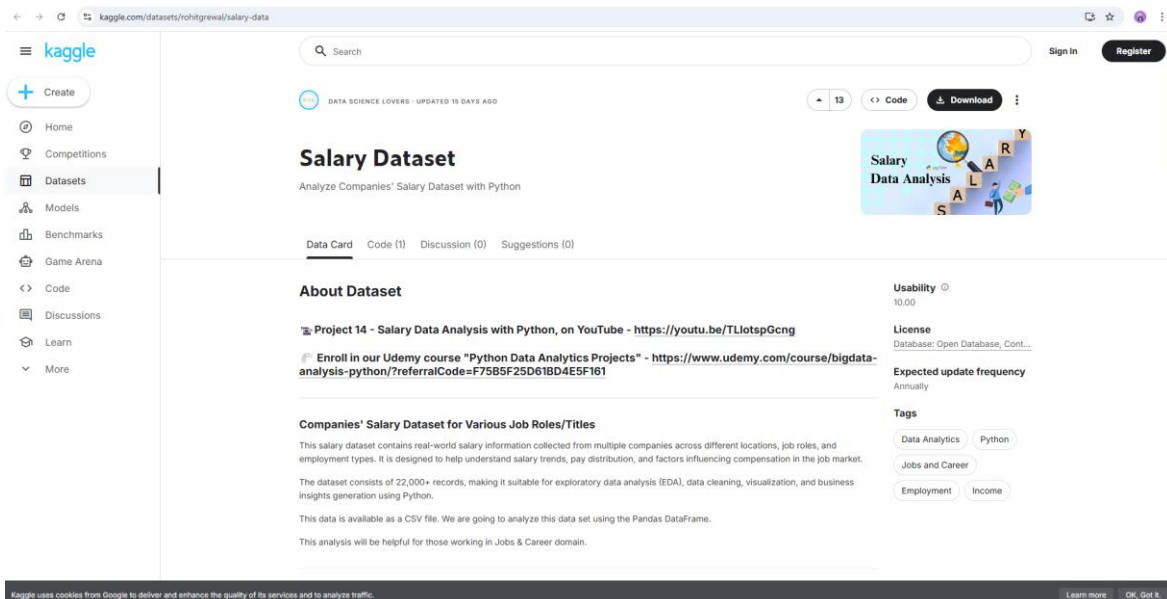
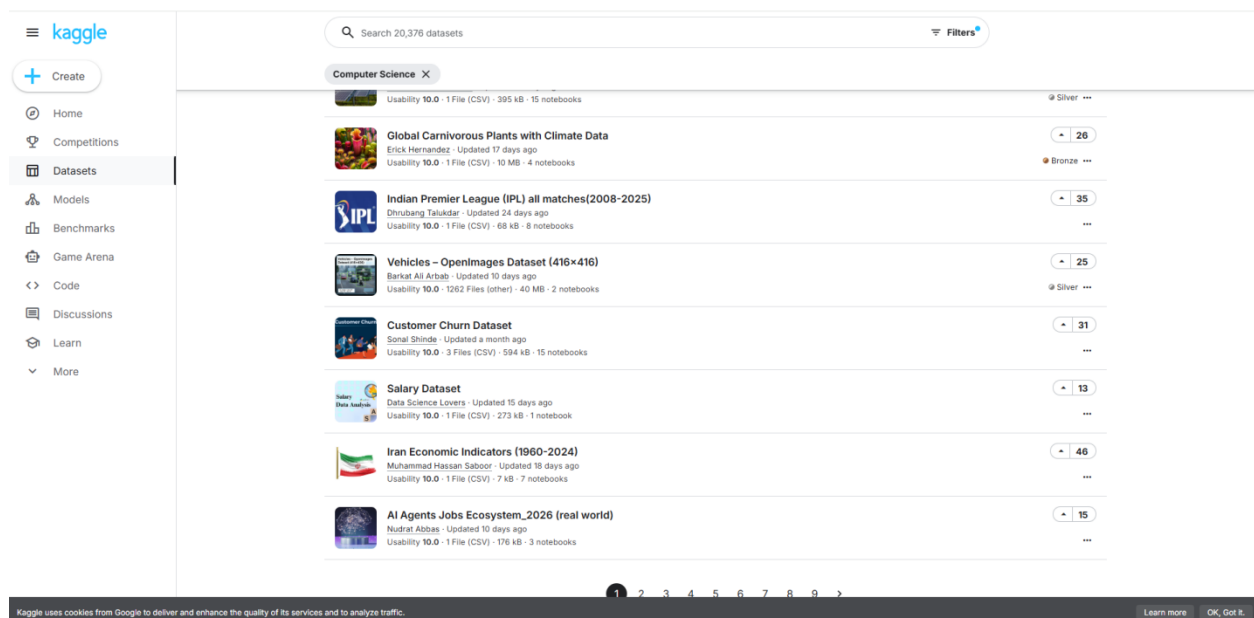
2 Categorical Attributes

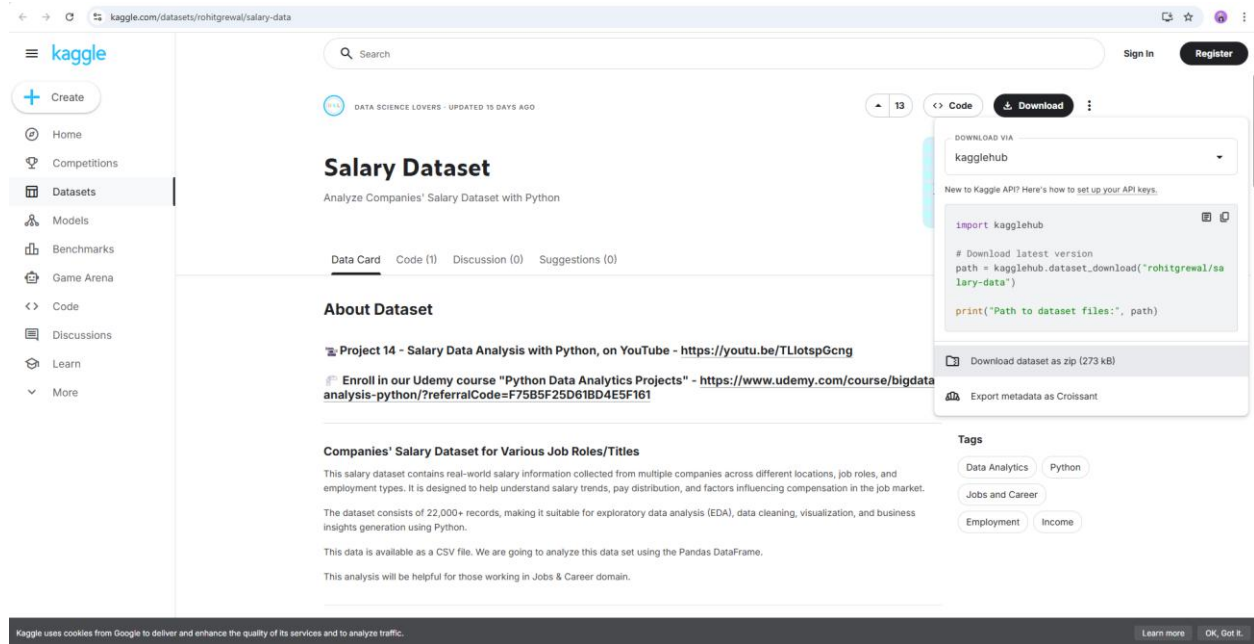
- Distinct categories
- Example: Gender, Department

3 Binary Attributes

- Two possible values
- Example: Yes/No, 0/1

How to download the dataset





TASK:

Write a Python program that does the following:

1. Create a list of integers entered by the user.
2. Using a **loop**, go through each number in the list.
3. For each number:
 - If the number is **even**, print "Even"
 - If the number is **odd**, print "Odd"
4. At the end, print:
 - The **total count of even numbers**
 - The **total count of odd numbers**

INPUT

Enter numbers separated by spaces: 3 10 7 8 12

OUTPUT

```
3 is Odd
10 is Even
7 is Odd
8 is Even
12 is Even
```

```
Total Even numbers: 3
Total Odd numbers: 2
```

Introduction to Knime

KNIME (Konstanz Information Miner) is an open-source data analytics, reporting, and integration platform. It is designed to help users build **data workflows visually**, without heavy coding.

What is KNIME Used For?

KNIME is commonly used for:

- Data analysis
- Machine learning
- Data cleaning & preprocessing
- Data visualization
- Predictive modeling
- Business intelligence

Task 1:

Working on Iris Dataset:

	A	B	C	D	E	F	G
1	Id	SepalLength	SepalWidth	PetalLength	PetalWidth	Species	
2	1	5.1	3.5	1.4	0.2	Iris-setosa	
3	2	4.9	3	1.4	0.2	Iris-setosa	
4	3	4.7	3.2	1.3	0.2	Iris-setosa	
5	4	4.6	3.1	1.5	0.2	Iris-setosa	
6	5	5	3.6	1.4	0.2	Iris-setosa	
7	6	5.4	3.9	1.7	0.4	Iris-setosa	
8	7	4.6	3.4	1.4	0.3	Iris-setosa	
9	8	5	3.4	1.5	0.2	Iris-setosa	
10	9	4.4	2.9	1.4	0.2	Iris-setosa	
11	10	4.9	3.1	1.5	0.1	Iris-setosa	
12	11	5.4	3.7	1.5	0.2	Iris-setosa	
13	12	4.8	3.4	1.6	0.2	Iris-setosa	
14	13	4.8	3	1.4	0.1	Iris-setosa	
15	14	4.3	3	1.1	0.1	Iris-setosa	
16	15	5.8	4	1.2	0.2	Iris-setosa	
17	16	5.7	4.4	1.5	0.4	Iris-setosa	
18	17	5.4	3.9	1.3	0.4	Iris-setosa	
19	18	5.1	3.5	1.4	0.3	Iris-setosa	
20	19	5.7	3.8	1.7	0.3	Iris-setosa	

Calculating Sum, Count, Min, Max, Average, Mode:

By using the following formulas we calculate all these:

=COUNT(A1:A151)

=SUM(A1:A151)

=AVERAGE(A1:A151)

=MODE(B2:B151)

=MIN(A1:A151)

=MAX(A1:A151)

11325	876.5	458.1	563.8	179.8
150	150	150	150	150
75.5	5.8433333	3.054	3.7586667	1.1986667
	5	3	1.5	0.2
1	4.3	2	1	0.1
150	7.9	4.4	6.9	2.5

To form a table:

CTRL + T

Create Table

×

Where is the data for your table?

=A\$1:\$F\$157

☒ My table has headers

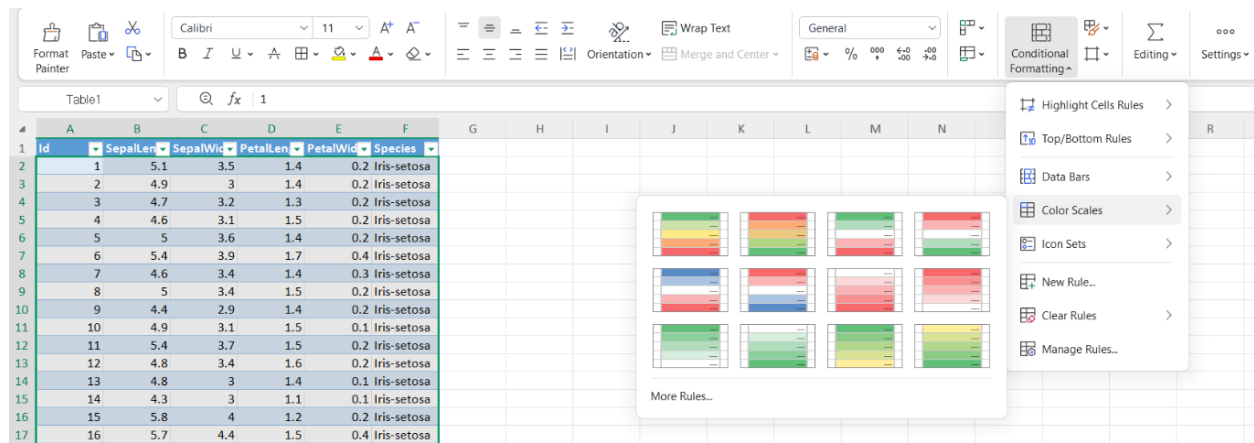
☒ Filter the table

OK

Cancel

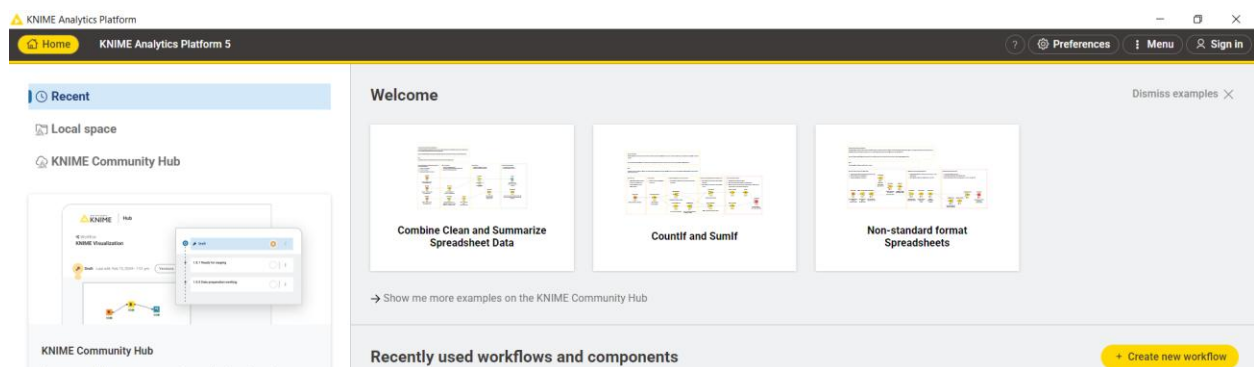
1	Id	SepalLen	SepalWid	PetalLen	PetalWid	Species
2	1	5.1	3.5	1.4	0.2	Iris-setosa
3	2	4.9	3	1.4	0.2	Iris-setosa
4	3	4.7	3.2	1.3	0.2	Iris-setosa
5	4	4.6	3.1	1.5	0.2	Iris-setosa
6	5	5	3.6	1.4	0.2	Iris-setosa
7	6	5.4	3.9	1.7	0.4	Iris-setosa
8	7	4.6	3.4	1.4	0.3	Iris-setosa
9	8	5	3.4	1.5	0.2	Iris-setosa
10	9	4.4	2.9	1.4	0.2	Iris-setosa

For highlighting tables:



1	Id	SepalLen	SepalWid	PetalLen	PetalWid	Species
2	1	5.1	3.5	1.4	0.2	Iris-setosa
3	2	4.9	3	1.4	0.2	Iris-setosa
4	3	4.7	3.2	1.3	0.2	Iris-setosa
5	4	4.6	3.1	1.5	0.2	Iris-setosa
6	5	5	3.6	1.4	0.2	Iris-setosa
7	6	5.4	3.9	1.7	0.4	Iris-setosa
8	7	4.6	3.4	1.4	0.3	Iris-setosa
9	8	5	3.4	1.5	0.2	Iris-setosa
10	9	4.4	2.9	1.4	0.2	Iris-setosa
11	10	4.9	3.1	1.5	0.1	Iris-setosa
12	11	5.4	3.7	1.5	0.2	Iris-setosa
13	12	4.8	3.4	1.6	0.2	Iris-setosa
14	13	4.8	3	1.4	0.1	Iris-setosa
15	14	4.3	3	1.1	0.1	Iris-setosa
16	15	5.8	4	1.2	0.2	Iris-setosa
17	16	5.7	4.4	1.5	0.4	Iris-setosa

Working on knime



Click on create new workflow:

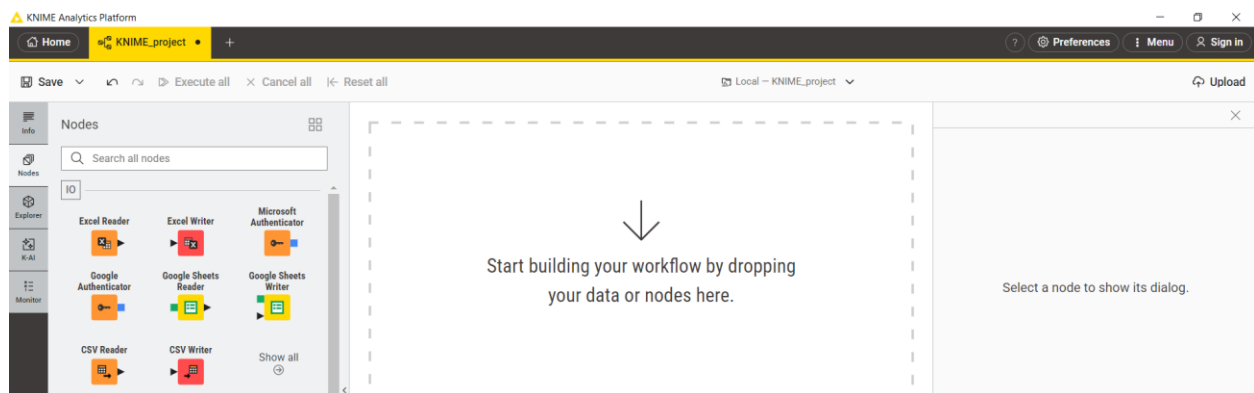
Create a new workflow

Workflow name

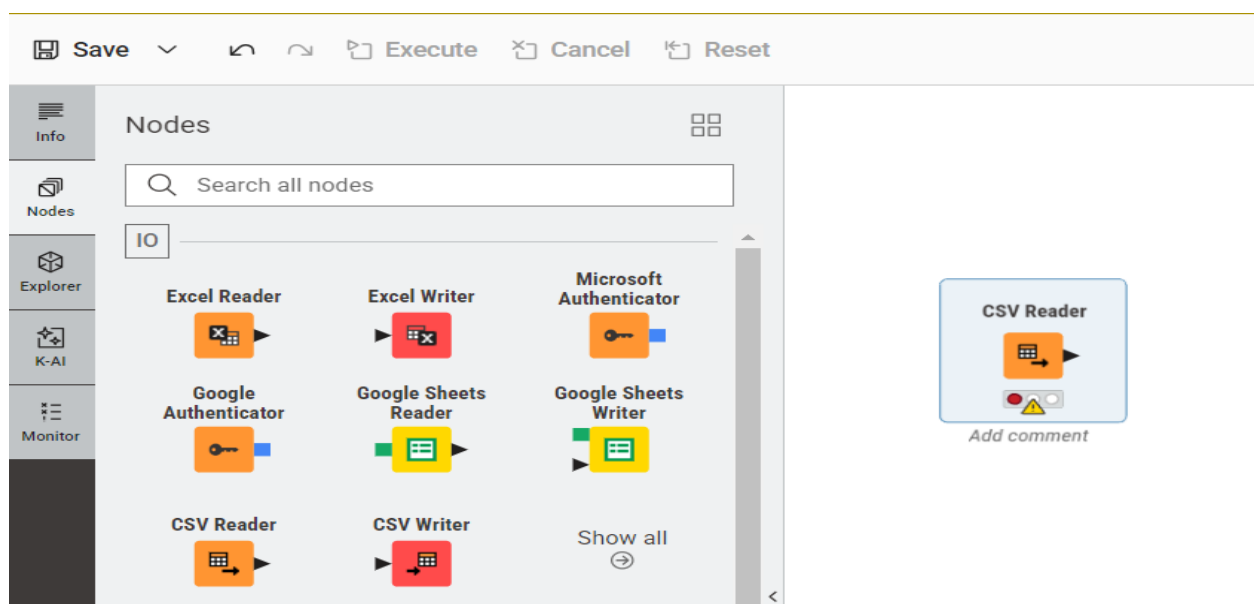
KNIME_Project

Cancel

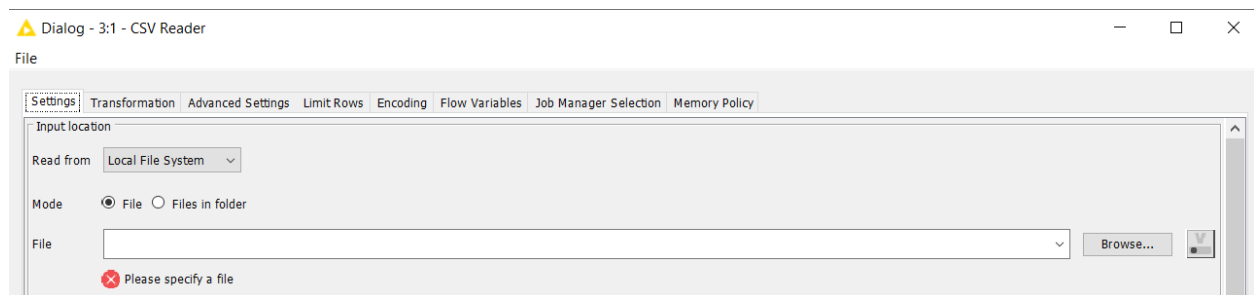
Create



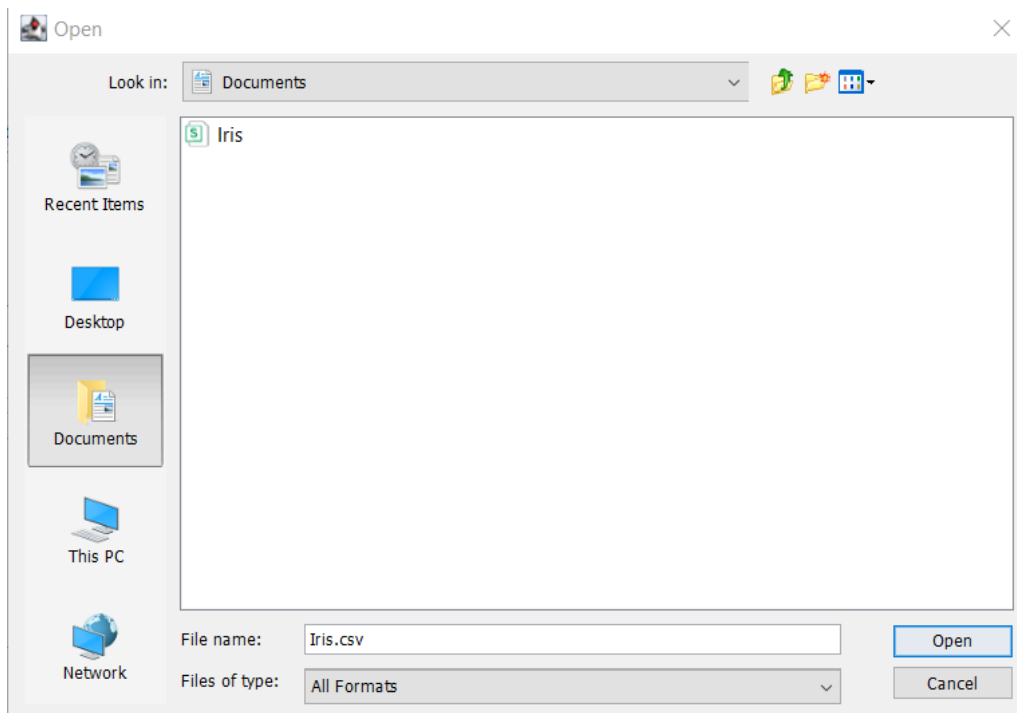
In Nodes section choose CSV reader:



Browse the file u want to upload:



Select the file and click open:



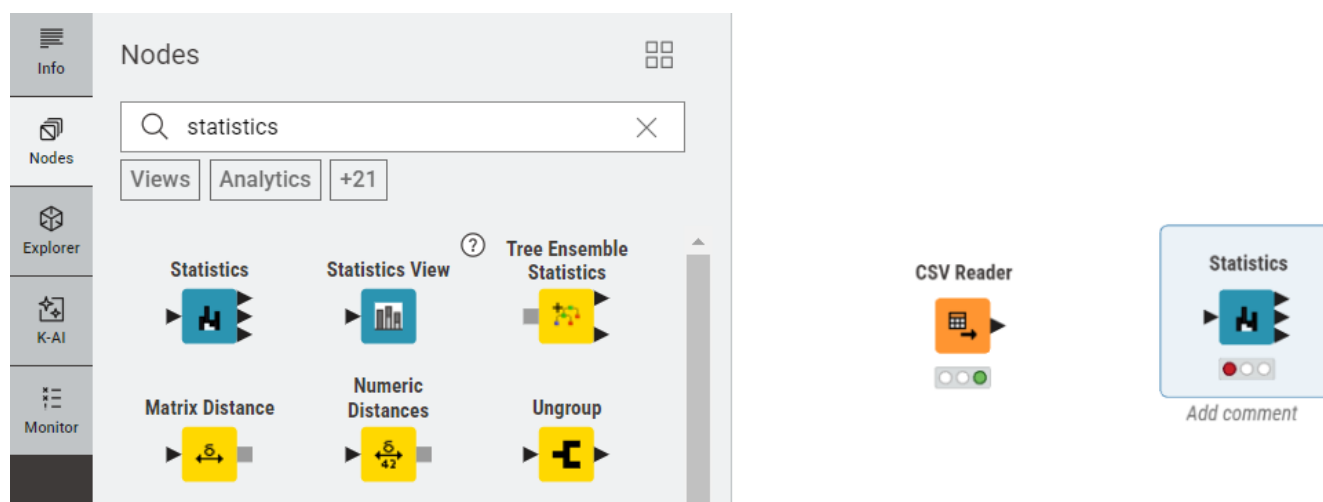
After clicking execute the CSV file will be uploaded in knime environment:

► 1: File Table Flow Variables

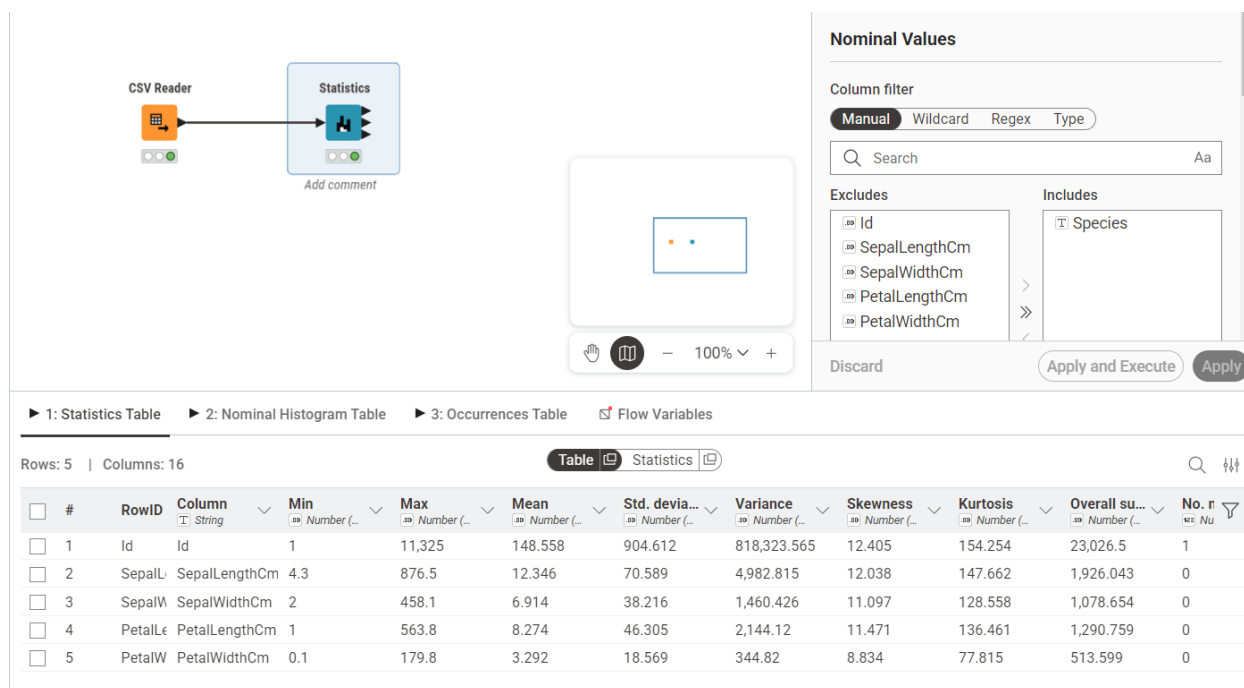
Rows: 156 | Columns: 6 Table Statistics

#	RowID	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
		Number (Float)	Number (Float)	Number (Float)	Number (Float)	Number (Float)	String
1	Row0	1	5.1	3.5	1.4	0.2	Iris-setosa
2	Row1	2	4.9	3	1.4	0.2	Iris-setosa
3	Row2	3	4.7	3.2	1.3	0.2	Iris-setosa
4	Row3	4	4.6	3.1	1.5	0.2	Iris-setosa
5	Row4	5	5	3.6	1.4	0.2	Iris-setosa
6	Row5	6	5.4	3.9	1.7	0.4	Iris-setosa

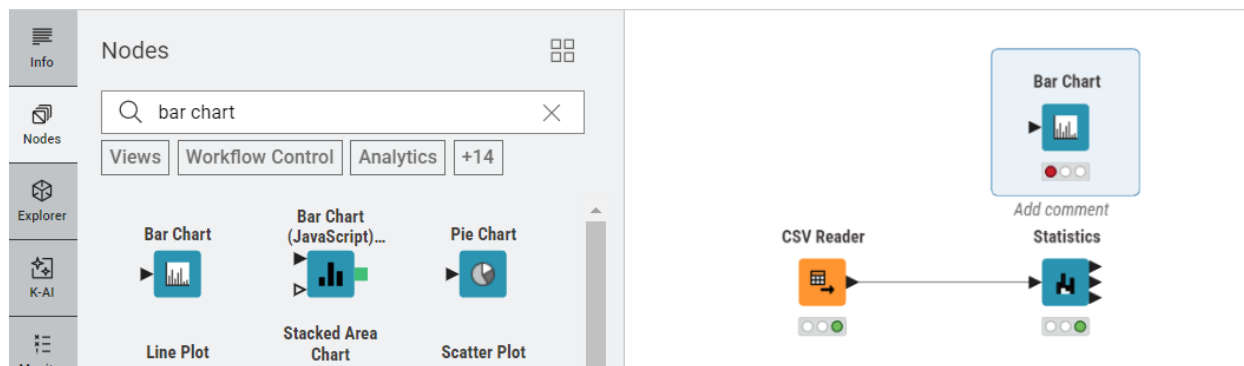
Now use statistic node:



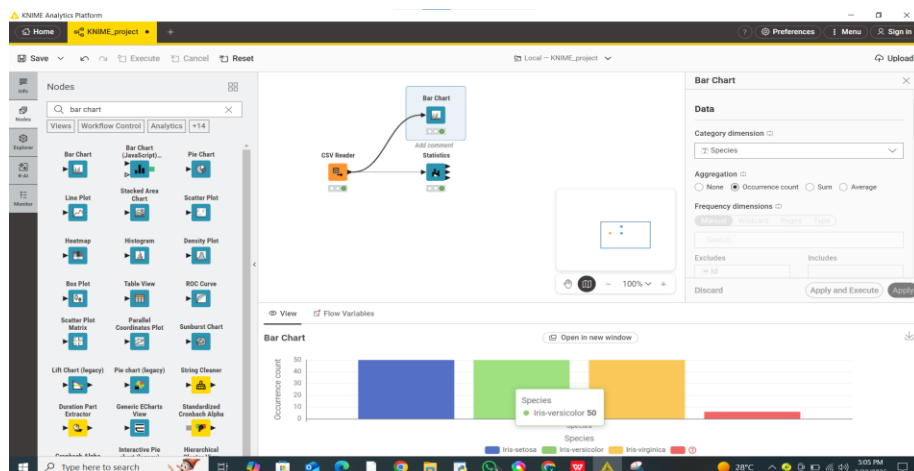
By joining it to the CSV node it will automatically display the mean, min, max, std dev etc.



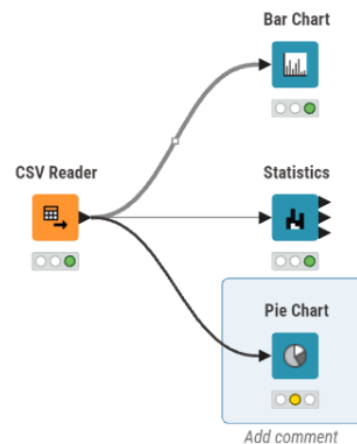
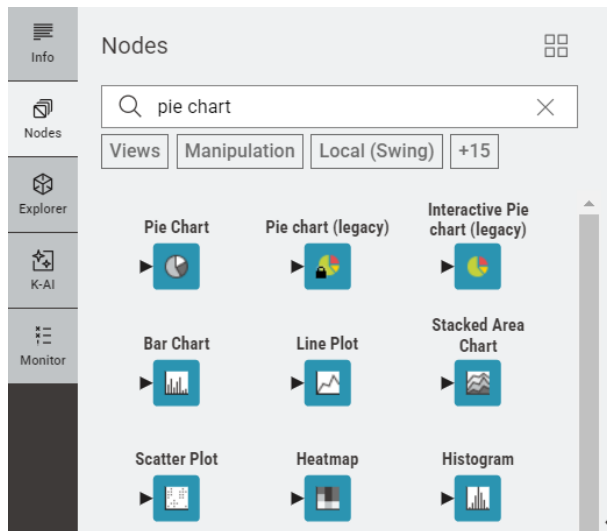
Now add barchart node:



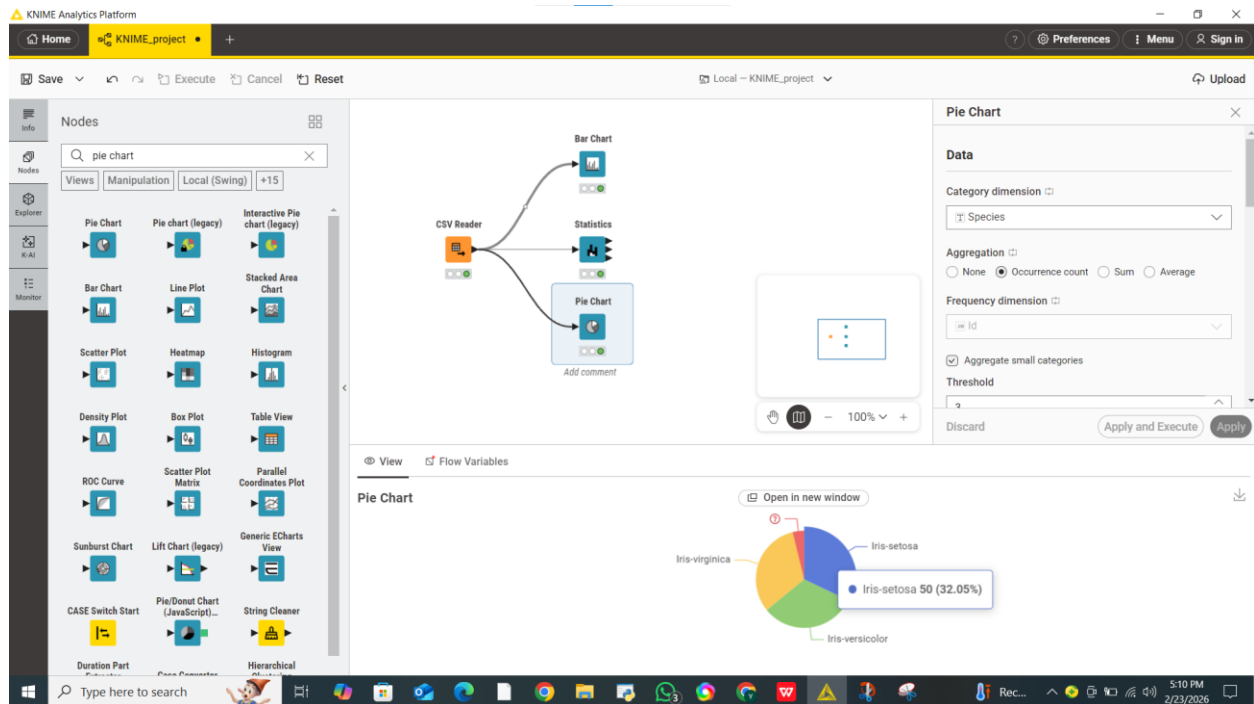
It will display the count of each specie:



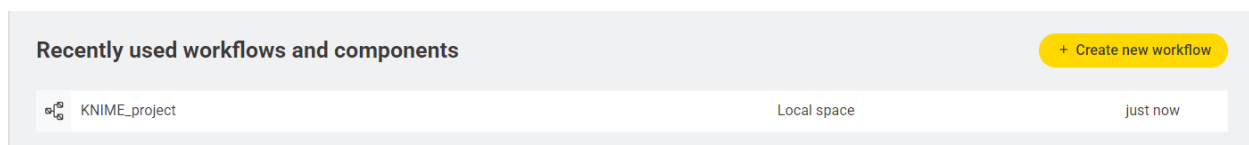
Now add the pie chart node:



It will display the percentage of each specie in pie chart:



Now save the workflow:



LAB-2

TOPICS

- Cross-Industry Standard Process for Data Mining (CRISP-DM)
- Customer Churn Prediction and Retention Analysis
- Visual Programming for Machine Learning using Orange Data Mining
- Classification Algorithms and Model Evaluation Metrics (Accuracy, ROC-AUC)

Overview of datasets

The Customer Churn Dataset contains historical data of e-commerce platform users, designed to predict whether a customer will stop using the service (churn).

The dataset consists of 3941 rows, 11 features, 0 metas.

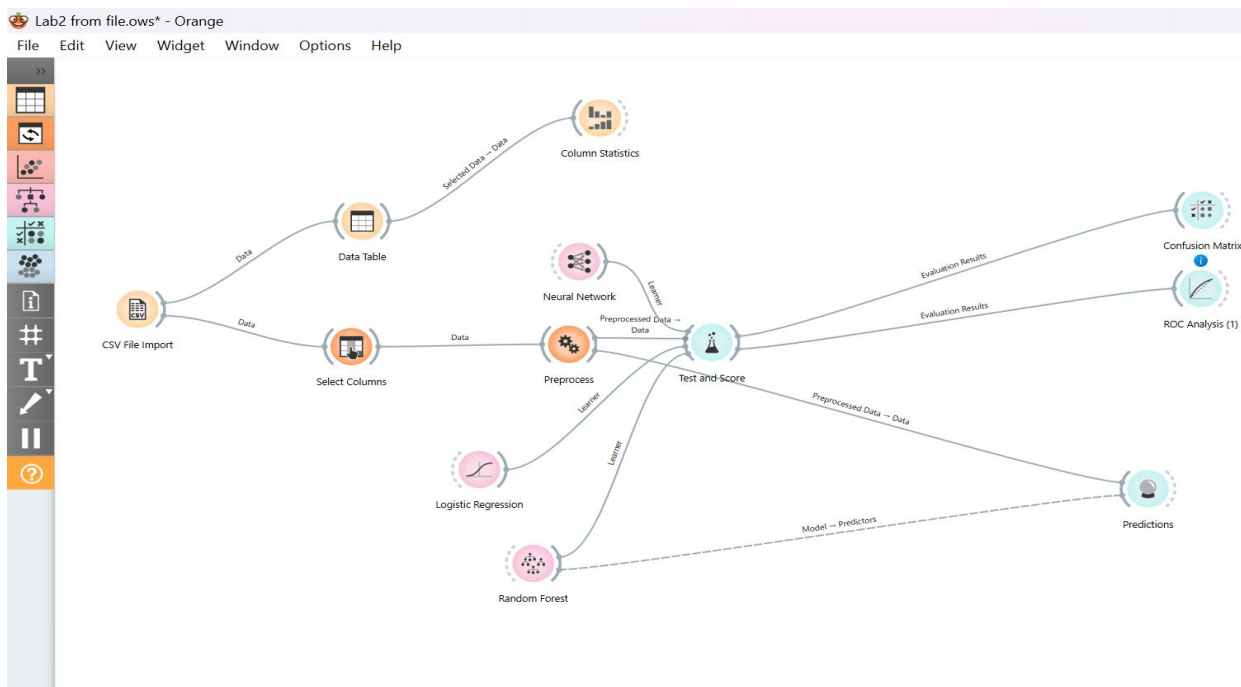
Key attributes typically include customer demographics, account information, and behavioral metrics (such as purchase frequency and tenure). The target variable is 'Churn', which is a binary classification indicating whether the customer stayed or left.

CLO:

Apply the CRISP-DM methodology to explore, preprocess, and model structured data, utilizing visual data mining tools to solve real-world business classification problems and evaluate model performance.

OBJECTIVE:

To develop a predictive machine learning model using Orange Data Mining that identifies e-commerce customers at high risk of churning, enabling targeted retention strategies. This lab will systematically walk through the CRISP-DM phases from Data Understanding to Model Evaluation.



System Methodology & Workflow Execution

The predictive modeling pipeline was constructed using a modular, visual programming approach in Orange Data Mining. Each component (widget) represents a specific stage of the CRISP-DM process:

Data Ingestion & Selection

- **CSV File Import:** Serves as the entry point where the E-commerce Churn dataset is loaded.
- **Select Columns:** This widget is used to define the **Target Variable** (Churn) and separate descriptive features from administrative data (Meta attributes like Customer IDs).
- **Data Table & Column Statistics:** These provide a real-time view of the raw data and summary statistics to ensure data integrity before modeling.

Preprocessing & Feature Engineering

- **Preprocess:** This is a critical mid-step where missing values are imputed and categorical data is numerically encoded. This ensures that the data is in the correct mathematical format for algorithms like Neural Networks and Logistic Regression.

Comparative Modeling

Three distinct learners are connected to the **Test and Score** widget to perform a side-by-side performance evaluation:

1. **Neural Network:** Captures deep, non-linear relationships using a multi-layer perceptron architecture.
2. **Logistic Regression:** Acts as a high-interpretability baseline model to identify linear correlations.
3. **Random Forest:** An ensemble method that combines multiple decision trees to provide robust, high-accuracy predictions.

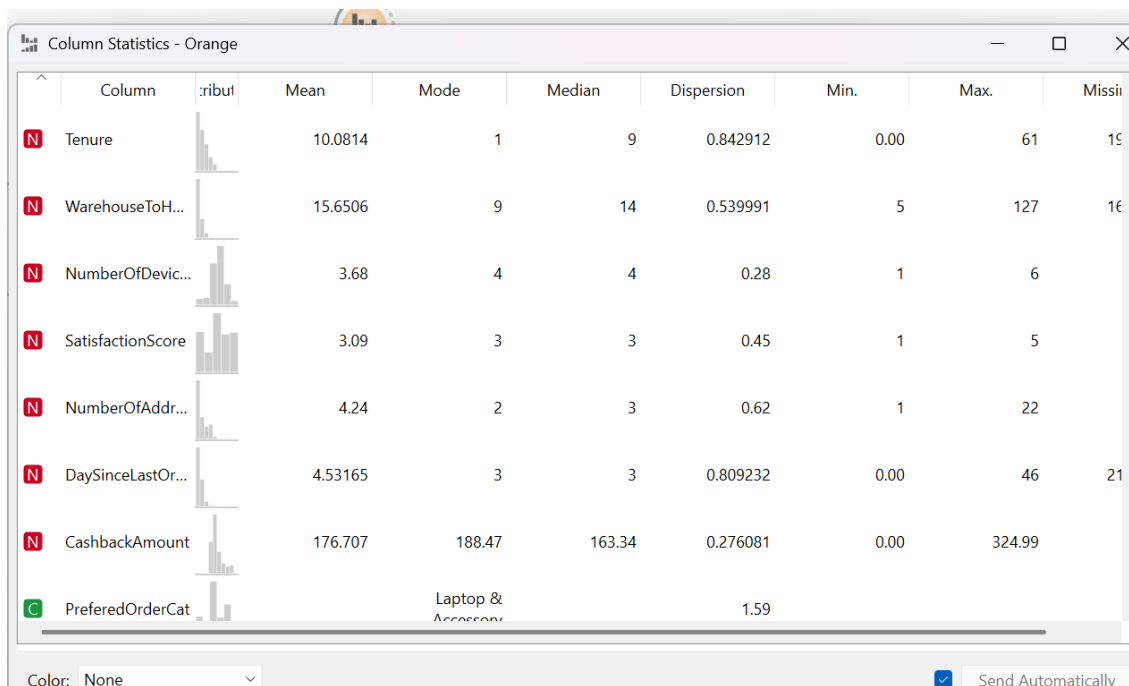
Evaluation & Decision Support

- **Test and Score:** Executes 10-fold cross-validation to provide unbiased metrics (AUC, CA, F1, Precision, and Recall).
- **Confusion Matrix & ROC Analysis:** These visualization widgets help in diagnosing model errors, specifically focusing on minimizing "False Negatives" (failing to identify a customer who is about to churn).

Predictive Deployment

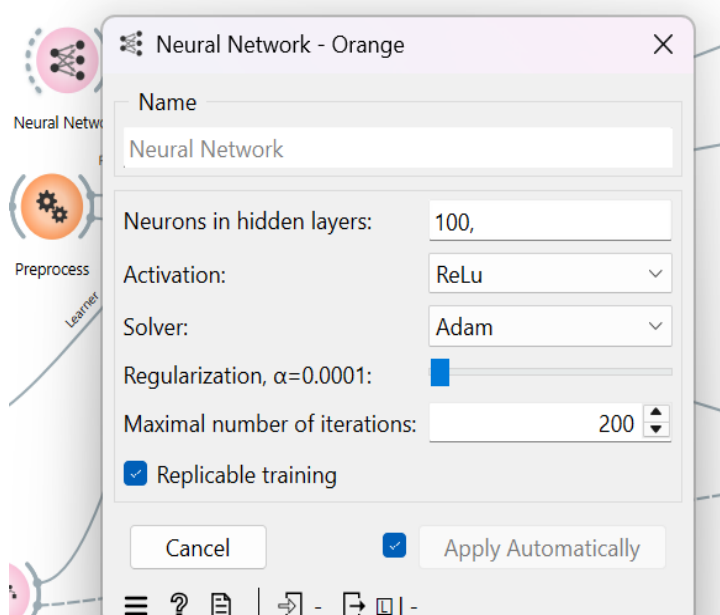
- **Predictions:** The final stage where the trained **Random Forest** model is applied to the processed data. This generates specific churn probability scores for individual customers, allowing for targeted business interventions.

COLUMN STATISTICS



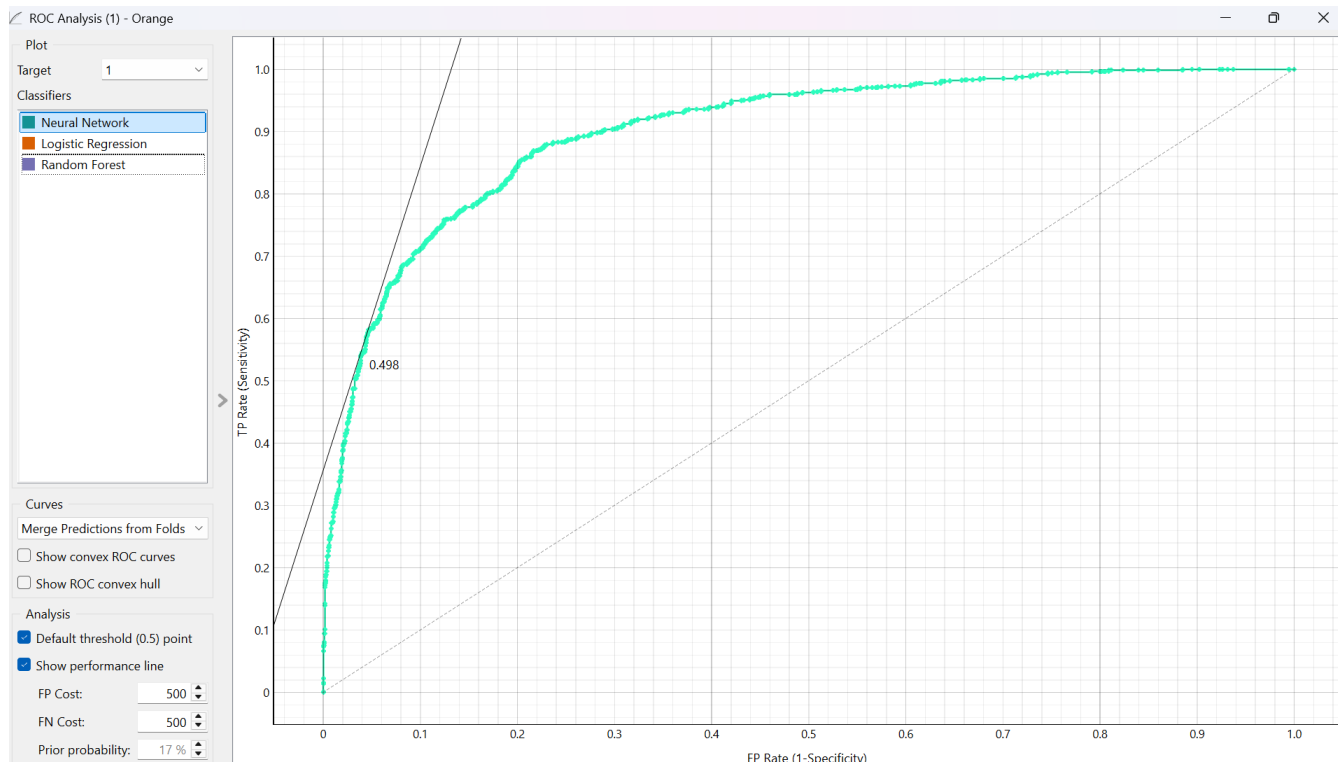
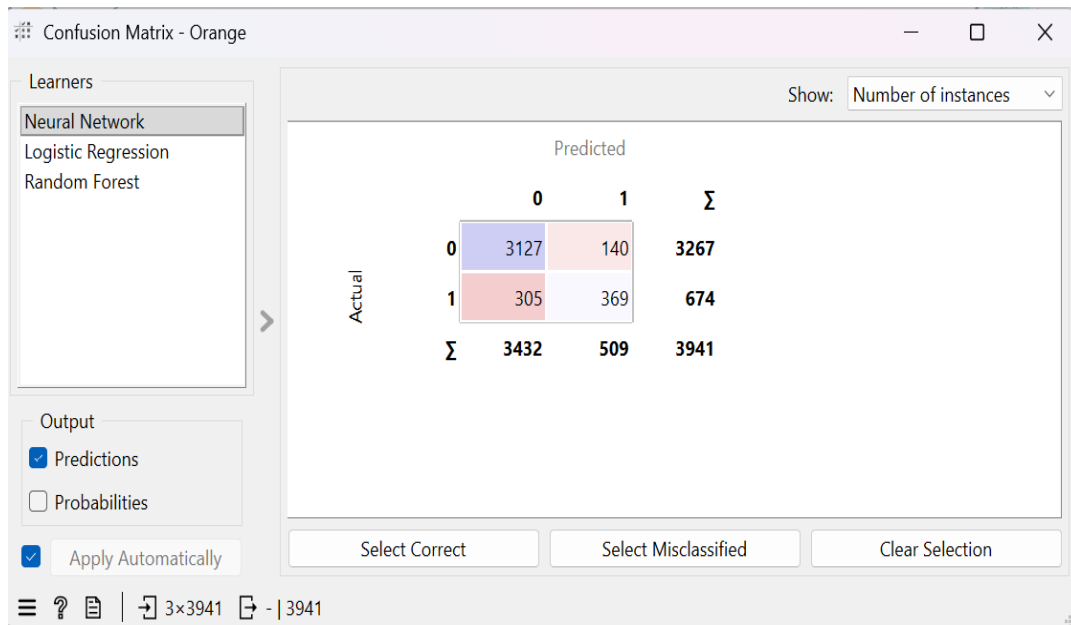
Neural Network using RELU

Neural Network: Captures deep, non-linear relationships using a multi-layer perceptron architecture.



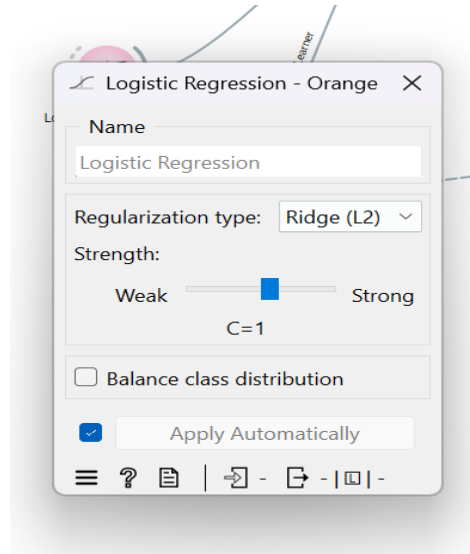
Confusion Matrix for Neural network & ROC Analysis

- **Confusion Matrix & ROC Analysis:** These visualization widgets help in diagnosing model errors, specifically focusing on minimizing "False Negatives" (failing to identify a customer who is about to churn).



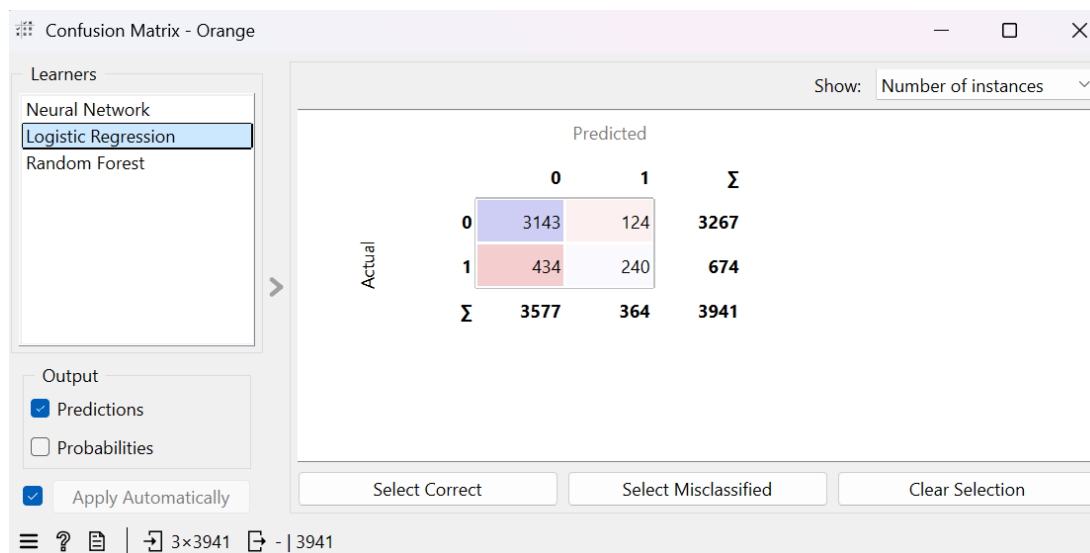
Logistic Regression

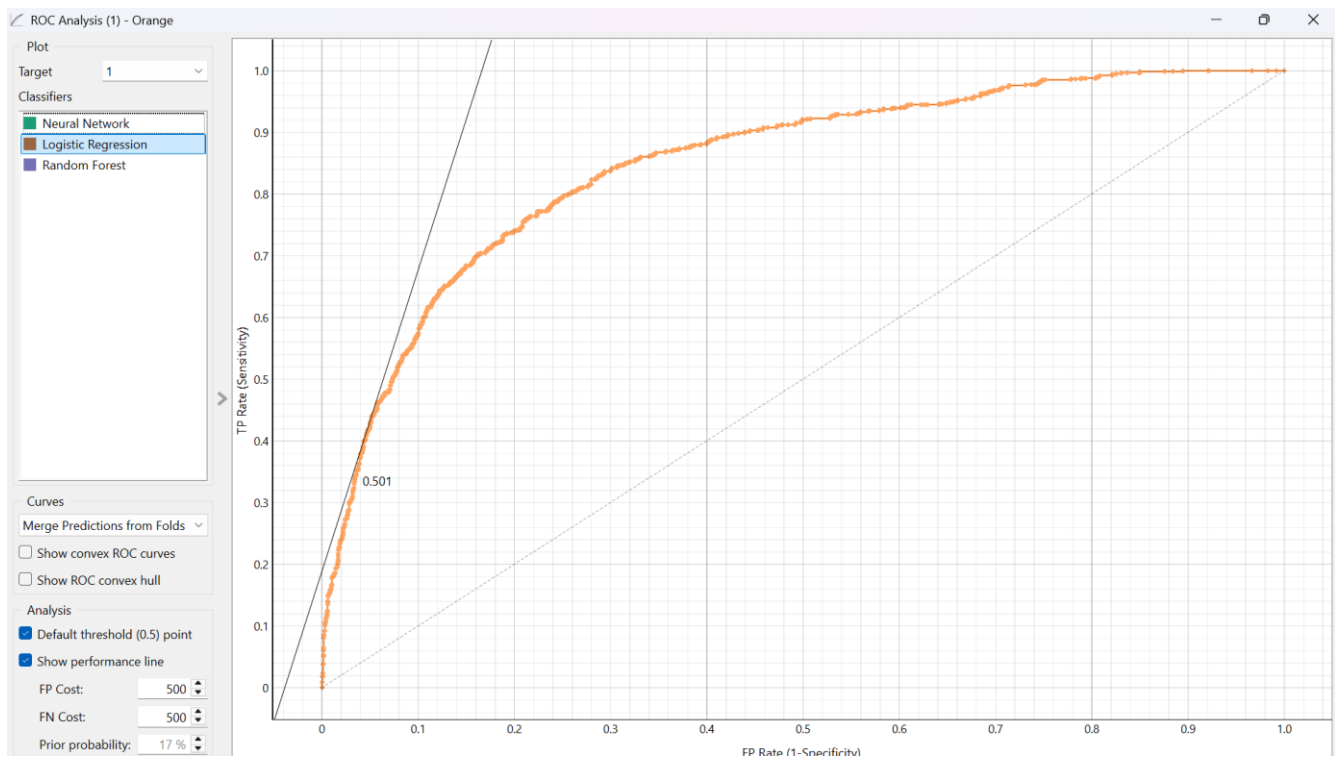
1. **Logistic Regression:** Acts as a high-interpretability baseline model to identify linear correlations.



Confusion Matrix for Logistic Regression & ROC Analysis

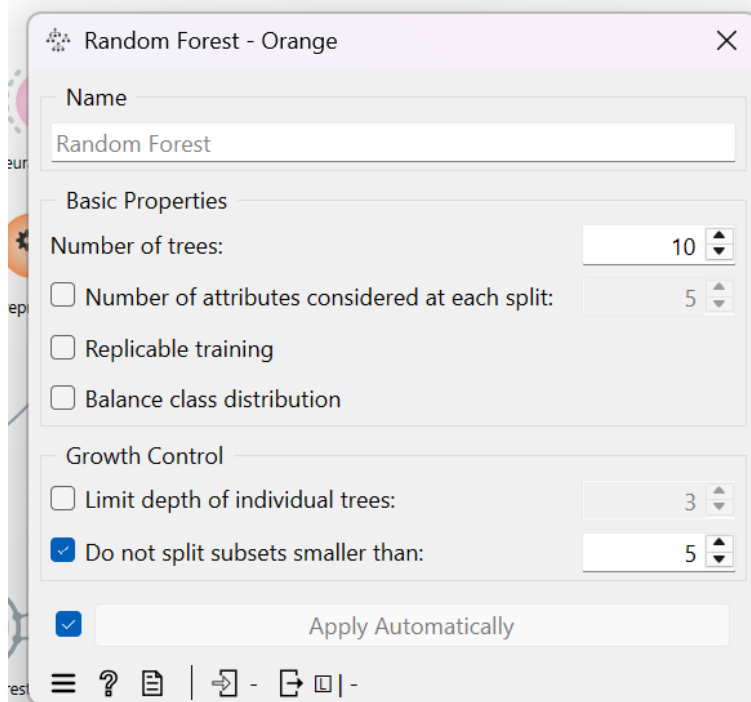
- **Confusion Matrix & ROC Analysis:** These visualization widgets help in diagnosing model errors, specifically focusing on minimizing "False Negatives" (failing to identify a customer who is about to churn).





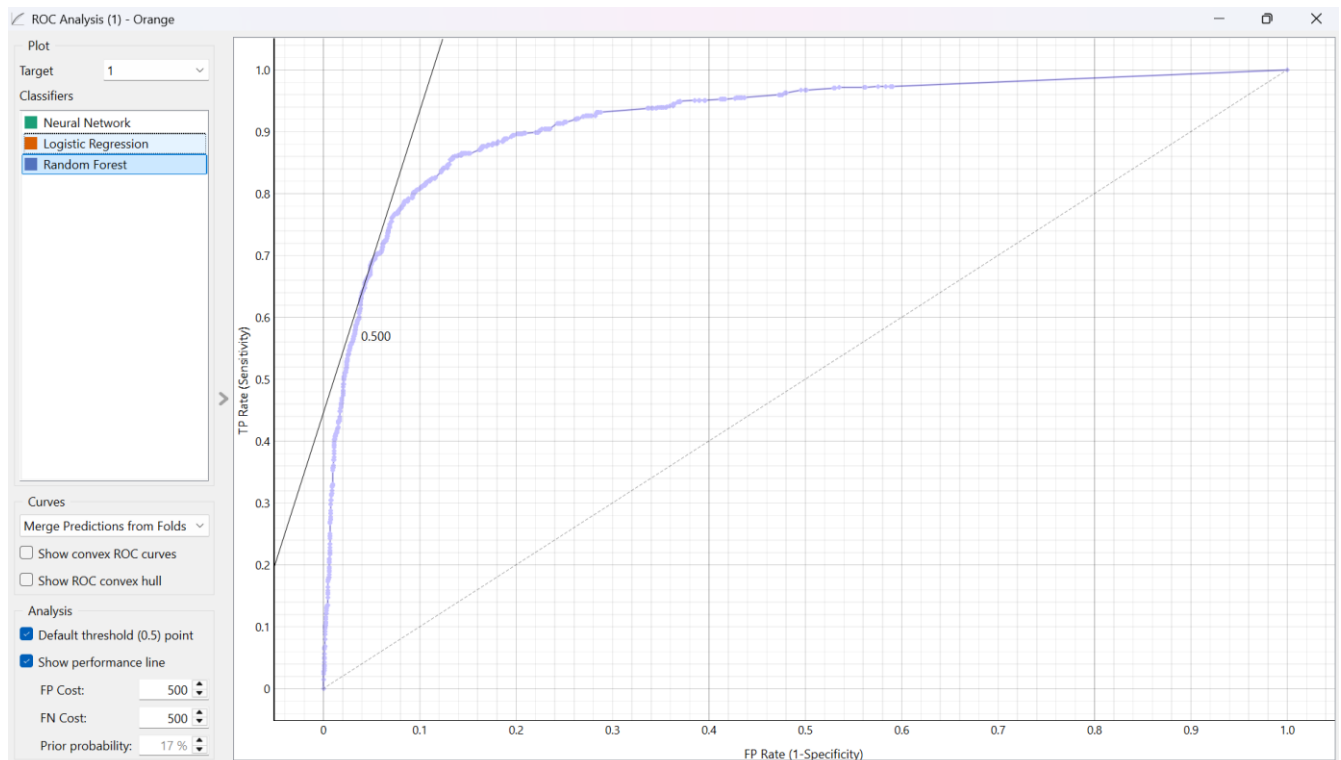
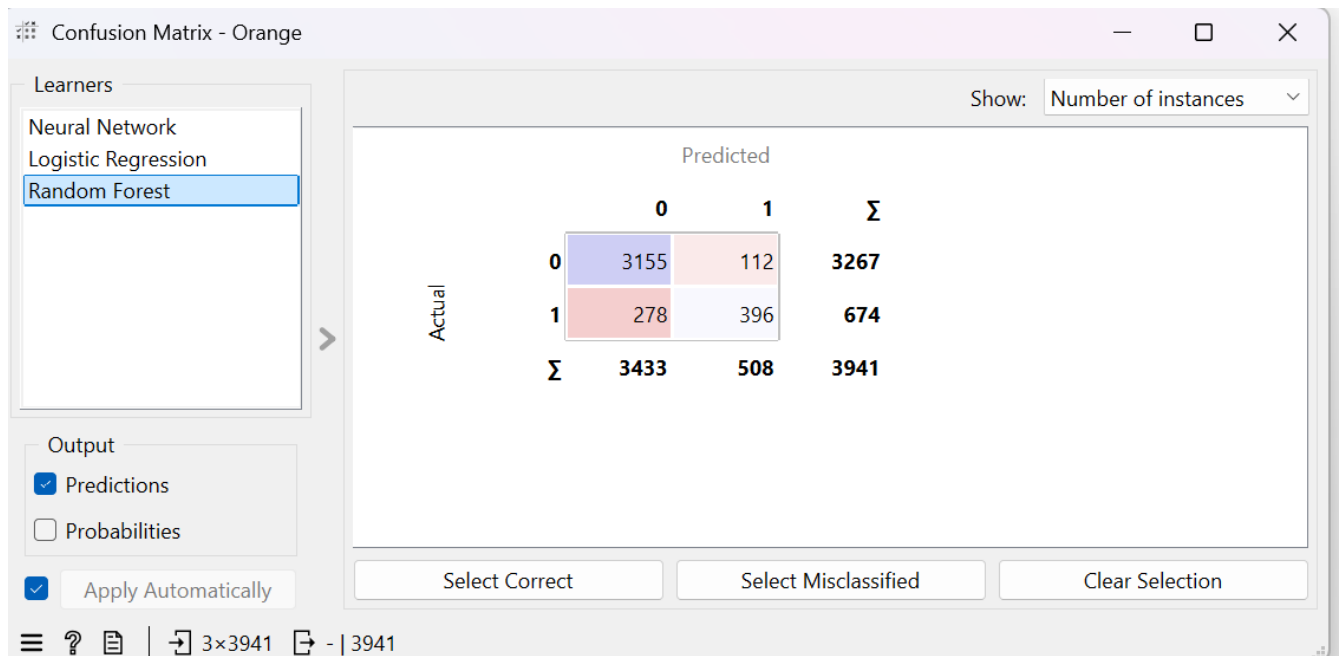
Random Forest

An ensemble method that combines multiple decision trees to provide robust, high-accuracy predictions.



Confusion Matrix for Logistic Regression & ROC Analysis

- **Confusion Matrix & ROC Analysis:** These visualization widgets help in diagnosing model errors, specifically focusing on minimizing "False Negatives" (failing to identify a customer who is about to churn).



Predictive Deployment

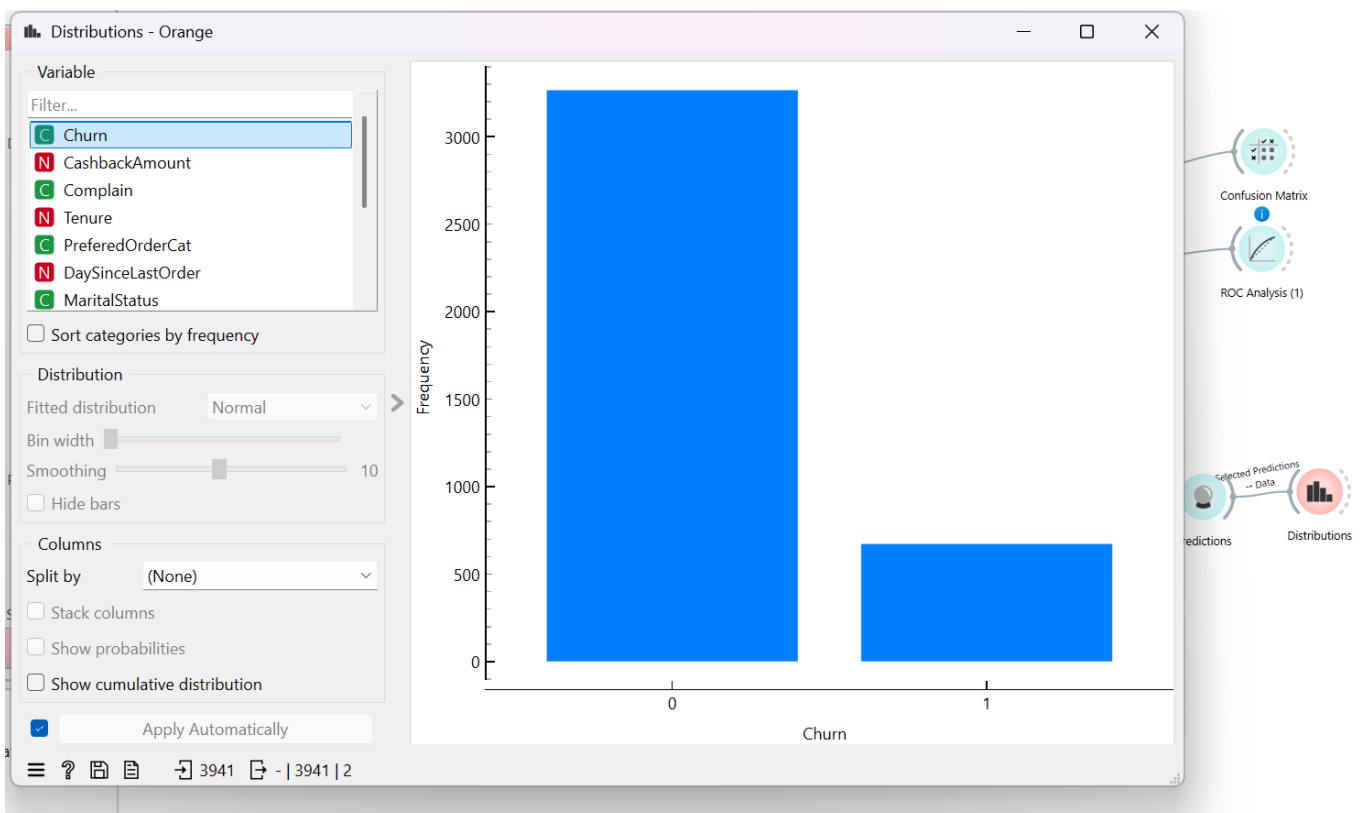
Predictions: The final stage where the trained **Random Forest** model is applied to the processed data. This generates specific churn probability scores for individual customers, allowing for targeted business intervention.

Predictions - Orange

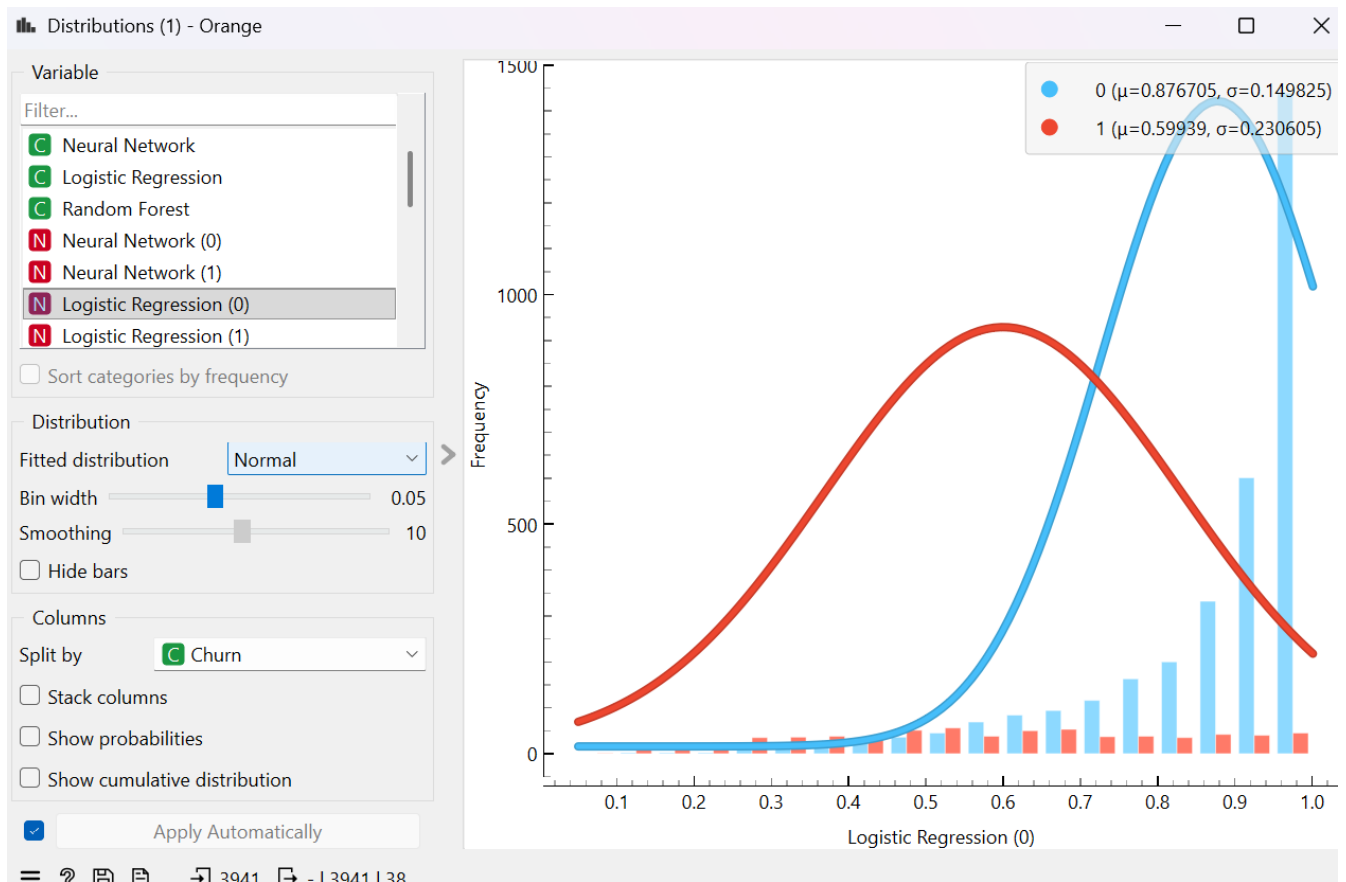
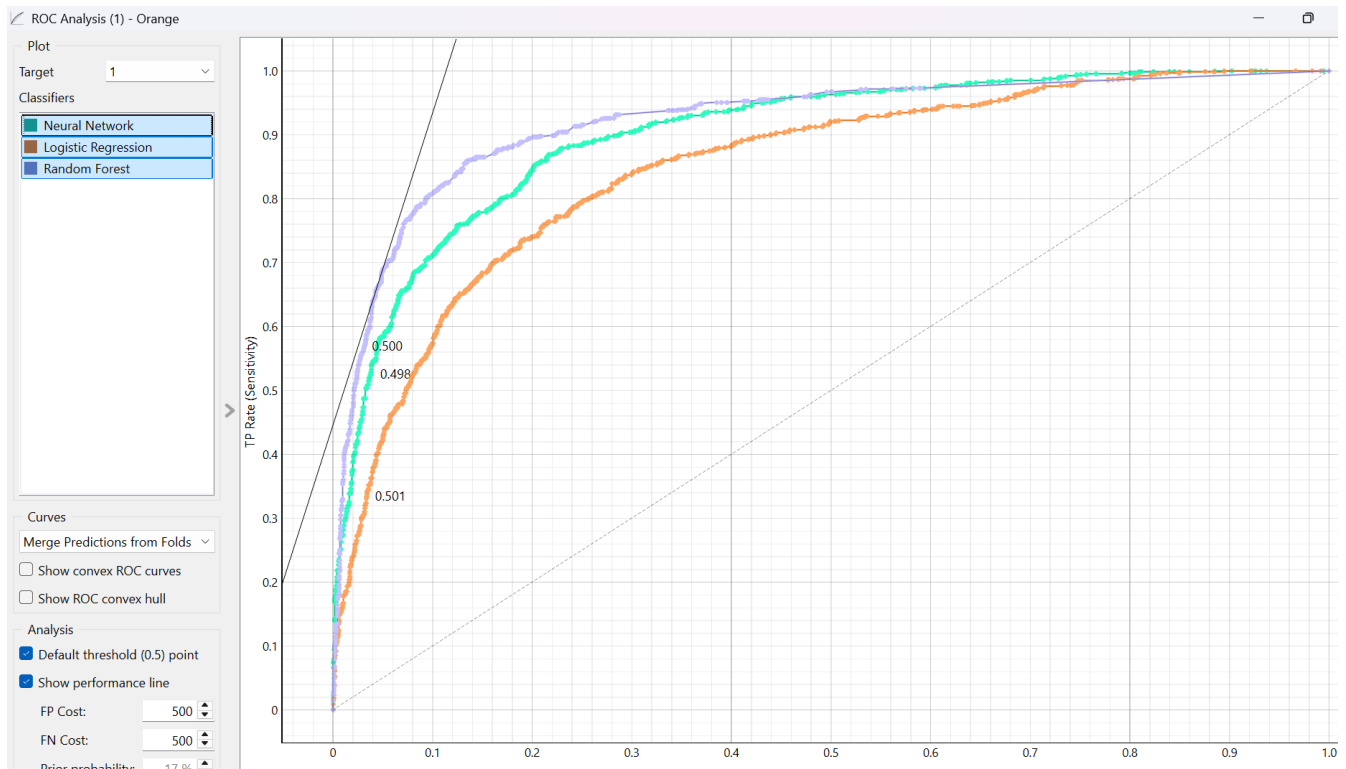
Show probabilities for: **Classes in data** ☒ Show classification errors Restore Original Order

	Churn	CashbackAmount	Complain	Tenure	PreferedOrderCat	DaySinceLastOrder	MaritalStatus	SatisfactionScore	berOfDeviceRegis	NumberofAddress	VarehouseToHom
0	143.32	0	0	0.578814	Laptop & Acces...	0.673096	Single	-0.064	0.316	-0.852	1.579592
0	129.29	0	0	-0.362614	Mobile	0.673096	Married	-1.511	0.316	-0.852	1.106285
0	168.54	0	0	1.990955	Laptop & Acces...	0.673096	Married	-1.511	-0.670	0.290	-0.313635
0	230.27	0	0	1.167206	Fashion	-0.0000	Divorced	-0.064	0.316	1.052	1.106285
0	322.17	0	0	2.343991	Others	0.945787	Single	0.660	0.316	1.432	-0.076981
0	152.81	0	0	-0.362614	Mobile Phone	1.763859	Divorced	-0.788	0.316	-0.852	0.041345
0	149.51	0	0	-1.068685	Mobile Phone	-0.690359	Divorced	1.384	2.289	-0.471	-0.076981
1	154.73	0	0	-1.068685	Mobile Phone	-0.963049	Single	1.384	0.316	-0.471	-0.550288
0	137.02	0	0	0.108100	Mobile Phone	-0.144977	Married	-0.064	0.316	-0.852	-0.431961
0	157.43	1	0	0.814171	Laptop & Acces...	-0.690359	Married	-1.511	-0.670	0.290	-1.023595
0	221.1	0	0	-0.480293	Fashion	-0.0000	Married	-0.064	2.289	0.290	1.224612
0	180.38	0	0	0.696492	Laptop & Acces...	-0.963049	Married	1.384	-1.656	1.052	0.514652
1	154.09	0	0	-1.068685	Mobile Phone	-0.417668	Divorced	-0.064	0.316	0.290	-1.023595
1	231.09	0	0	-1.068685	Fashion	-0.0000	Divorced	-0.064	1.303	1.052	-1.023595
0	163.92	0	0	-0.480293	Laptop & Acces...	-0.690359	Married	1.384	-0.670	-0.471	-0.195308
0	193.15	0	0	-0.009579	Laptop & Acces...	-0.963049	Single	-0.064	0.316	1.432	-0.905268
0	121.68	1	0	0.0000	Mobile	-0.417668	Married	-1.511	1.303	-1.233	0.632979
1	204.78	0	0	-0.244936	Fashion	0.400405	Divorced	-0.788	0.316	-0.471	0.751305
0	259.96	1	0	1.049528	Grocery	1.763859	Married	-1.511	0.316	-0.852	1.342939
1	127.16	0	0	-1.186364	Mobile	-1.235740	Single	0.660	-2.643	-0.852	0.277999
1	191.96	0	0	-1.068685	Laptop & Acces...	-0.690359	Single	1.384	1.303	1.052	2.052899
0	153.01	0	0	-0.833328	Laptop & Acces...	-0.690359	Married	-0.064	-0.670	1.813	0.751305
0	126.7	1	0	2.343991	Mobile	-1.235740	Married	1.384	-2.643	0.290	-0.786941
0	189.98	0	0	-0.597971	Fashion	0.673096	Single	-1.511	-0.670	-0.852	-0.195308
0	205.95	0	0	-0.127257	Fashion	2.581932	Single	0.660	0.316	-0.852	-0.431961

☒ Show performance scores Target class: (Average over classes)



Comparing ALL Three models



LAB-3

TOPICS:

1. Data Cleaning and Preprocessing
2. Handling Missing Values & Noise Filtering
3. Removing Duplicates and Quality Assessment
4. Software Introduction: Altair AI Studio (RapidMiner)

CLO: Understand data mining concepts, preprocessing, statistics, and trends. By doing this course we will be able to work on real industry level projects in future and our basics and foundation of studies would be good.

OBJECTIVE:

1. Understand the necessity of cleaning raw healthcare datasets before analysis.
2. Use Altair AI Studio (RapidMiner) as a data preparation environment.
3. Identify and handle missing values, duplicates, and noisy data.
4. Evaluate data improvement by calculating % Missing Reduced, Duplicate Removal %, and Data Quality Scores.

REQUIRED SOFTWARE / TOOLS:

- Altair AI Studio (formerly RapidMiner Studio)
- Healthcare Dataset (diabetic_data.csv)
- Computer / Laptop

INTRODUCTION TO DATA CLEANING: Data Cleaning is an essential step in the data mining process that involves identifying and correcting (or removing) errors, inconsistencies, and inaccuracies within a dataset. Real-world data, especially healthcare data, is often incomplete or noisy.

Key Preprocessing Steps:

- **Handling Missing Values:** Replacing blank or unknown data (like ?) with standard values (mean, median) or removing affected rows/columns.
- **Duplicate Removal:** Identifying and removing redundant records (e.g., repeat hospital visits for the same patient) to prevent skewed analysis.
- **Noise Filtering:** Removing outliers or impossible values that do not make logical sense within the dataset.

SOFTWARE INTRODUCTION – ALTAIR AI STUDIO: What is Altair AI Studio? Altair AI Studio (formerly RapidMiner) is a powerful, visual workflow designer for data science and machine learning. It allows users to build complex data pipelines using a drag-and-drop interface without needing to write code.

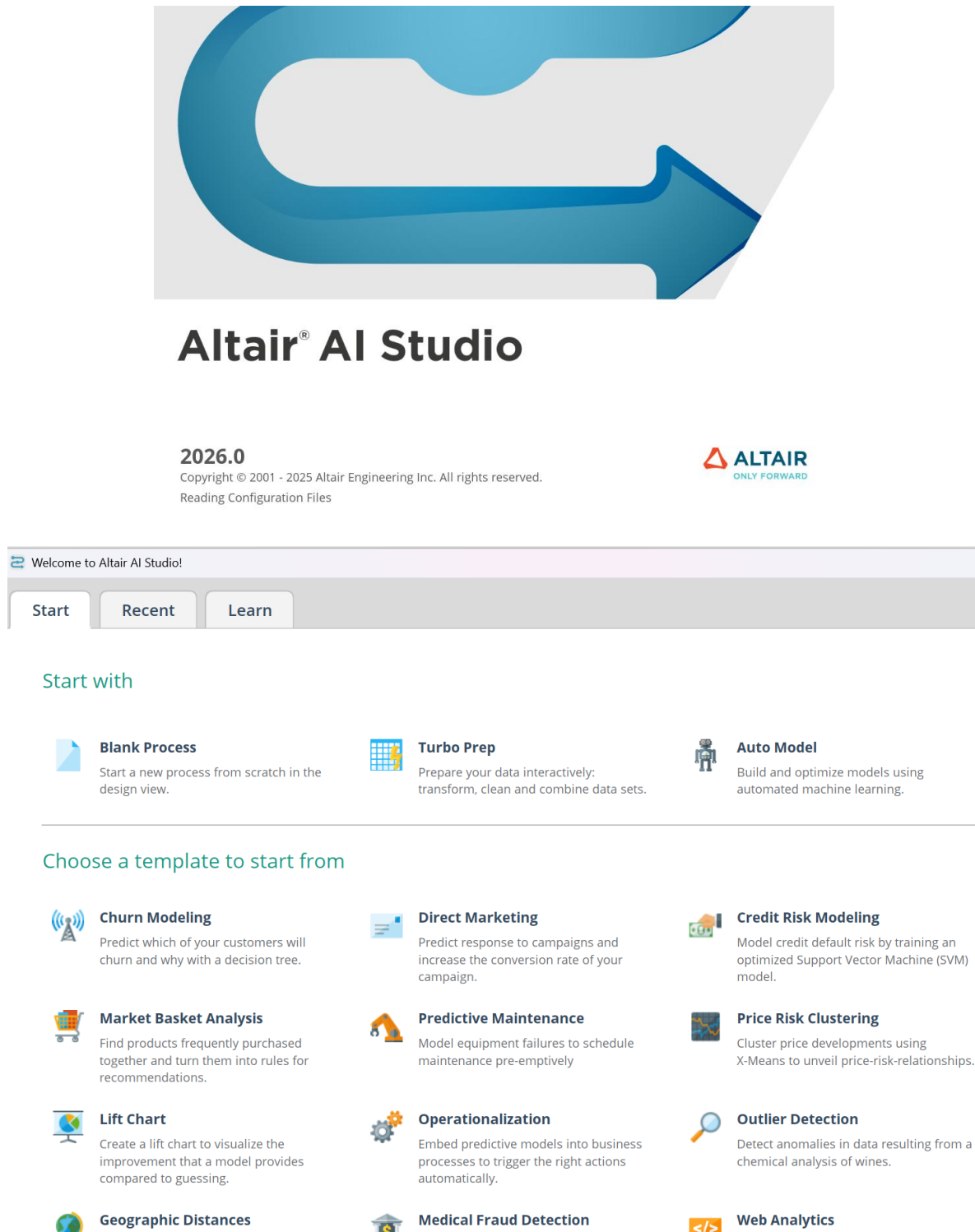
Features:

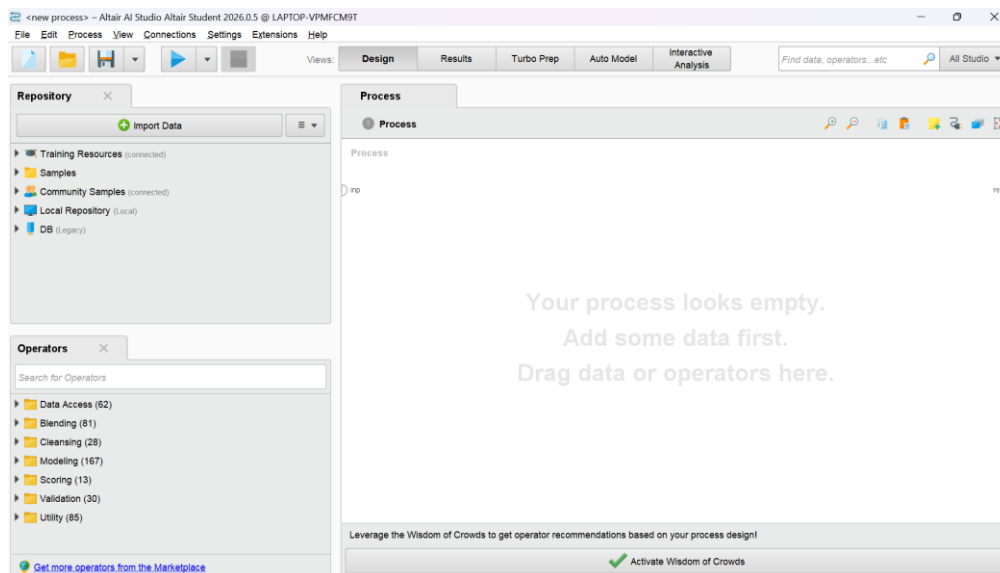
- Visual, code-free workflow design (Canvas and Operators)
- Turbo Prep feature for automated Data Quality scoring
- Built-in operators for handling missing values, filtering, and joining datasets
- Easy calculation of statistics and data distributions

Phase 1: Getting Your Data In

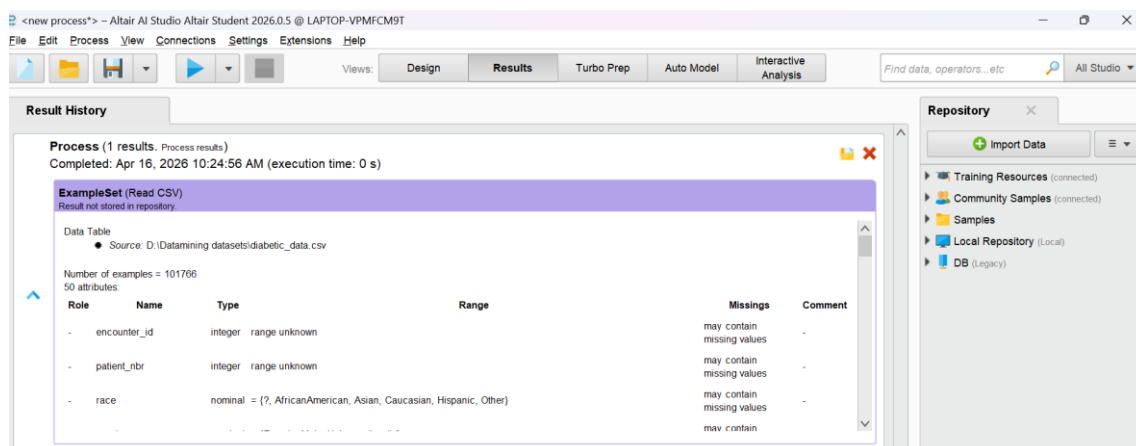
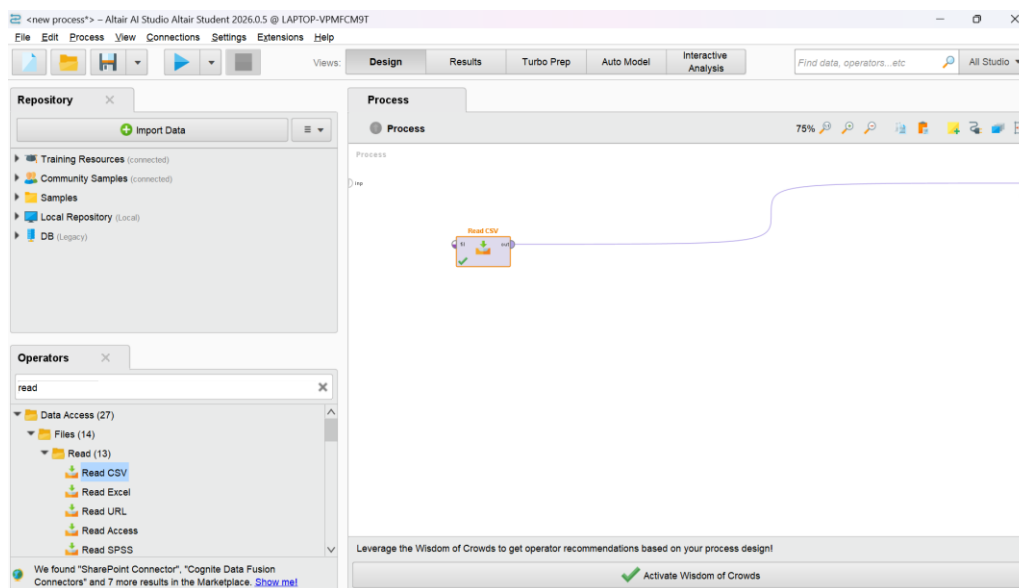
Before we clean, we need to load the data and check its initial "messy" state so you can calculate your percentages later.

1. Start a New Process: Open AI Studio and click Blank Process. You will see a large white space in the middle. This is your "Canvas."





Now we are loading our dataset



<new process*> - Altair AI Studio Altair Student 2026.0.5 @ LAPTOP-VPMFECM9T

File Edit Process View Connections Settings Extensions Help

Views: Design Results Turbo Prep Auto Model Interactive Analysis

filter All Studio

Result History

ExampleSet (Remove Duplicates) ExampleSet (Read CSV)

Open in Turbo Prep Auto Model Interactive Analysis

Filter (101,766 / 101,766 examples): all

Row No.	encounter_id	patient_nbr	race	gender	age	weight	admission_t...	discharge
1	2278392	8222157	Caucasian	Female	[0-10]	?	6	25
2	149190	55629189	Caucasian	Female	[10-20]	?	1	1
3	64410	86047875	AfricanAmeric...	Female	[20-30]	?	1	1
4	500364	82442376	Caucasian	Male	[30-40]	?	1	1
5	16680	42519267	Caucasian	Male	[40-50]	?	1	1
6	35754	82637451	Caucasian	Male	[50-60]	?	2	1
7	55842	84259809	Caucasian	Male	[60-70]	?	3	1
8	63768	114882984	Caucasian	Male	[70-80]	?	1	1
9	12522	48330783	Caucasian	Female	[80-90]	?	2	1
10	15738	63555939	Caucasian	Female	[90-100]	?	3	3
11	28236	89869032	AfricanAmeric...	Female	[40-50]	?	1	1
12	36900	77391171	AfricanAmeric...	Male	[60-70]	?	2	1
13	40926	85504905	Caucasian	Female	[40-50]	?	1	3
14	42570	77586282	Caucasian	Male	[80-90]	?	1	6
15	62256	49726791	AfricanAmeric...	Female	[60-70]	?	3	1

ExampleSet (101,766 examples, 0 special attributes, 50 regular attributes)

Repository

Import Data

BEFORE USING---

customer data

decision tree

the model

the model

validation

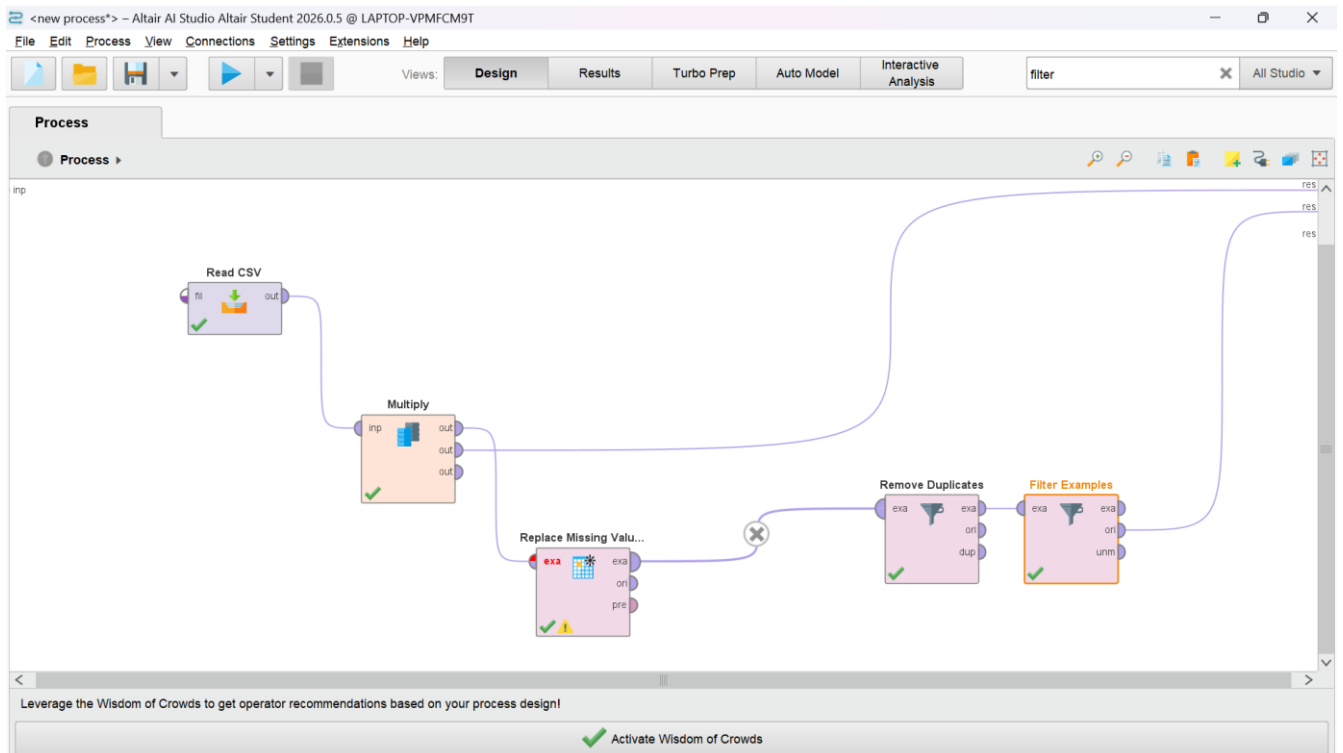
selection

eter optimization

atic Model Selection and Optimization

ality Reduction

ailure Remove Duplicates



Altair AI Studio Altair Student 2026.0.5 @ LAPTOP-VPMF0CM9T

File Edit Process View Connections Settings Extensions Help

Views: Design Results **Turbo Prep** Auto Model Interactive Analysis

filter All Studio

Turbo Prep

Generate

diabetic_data

Generate a new column below. You can type a formula or drag in columns from the left and functions from the right. Update the preview and - if all is correct - commit the result.

COMMIT GENERATE CLEAR ALL CANCEL

Name:

Please enter a name for the new column.

Formula:

Type a formula or drag columns and functions here

Please build a formula.

UPDATE PREVIEW

encounter_id	patient_nbr	race	gender
Number	Number	Category	Category
2278392	8222157	Caucasian	Female
149190	55629189	Caucasian	Female
64410	86047875	AfricanAmerican	Female

101,766 rows - 50 columns (37 nominal, 13 numerical)

Functions:

Logical

Comparison

Text Information

Text Transformation

Mathematical Functions

Statistical Functions

Constants:

Basic Constants

TRUE

FALSE

Data Quality Score : automatically calculates a "Quality" score for you, which is perfect for your requirement. Document the initial "Quality" score in Turbo Prep, run your cleaning process, and then check the final "Quality" score to show the improvement you made

Altair AI Studio Altair Student 2026.0.5 @ LAPTOP-VPMF0CM9T

File Edit Process View Connections Settings Extensions Help

Views: Design Results **Turbo Prep** Auto Model Interactive Analysis

filter All Studio

Turbo Prep

Data Sets

+ LOAD DATA

diabetic_data

//Local Repository/diabetic_data

Rows: 101,766

Columns: 50

Last Change: None

diabetic_data

Add new data sets on the left. Details for the selected data are shown below. You can change the data with the following actions.

TRANSFORM CLEANSE GENERATE PIVOT MERGE

encounter_id	patient_nbr	race	gender	age	weight	admission_ty...	discharge_dis...	admission_...
Number	Number	Category	Category	Category	Category	Number	Number	Number
2278392	8222157	Caucasian	Female	[0-10]	?	6	25	1
149190	55629189	Caucasian	Female	[10-20]	?	1	1	7
64410	86047875	AfricanAmerican	Female	[20-30]	?	1	1	7
500364	82442376	Caucasian	Male	[30-40]	?	1	1	7
16680	42519267	Caucasian	Male	[40-50]	?	1	1	7
35754	82637451	Caucasian	Male	[50-60]	?	2	1	2
55842	84259809	Caucasian	Male	[60-70]	?	3	1	2
63768	114882984	Caucasian	Male	[70-80]	?	1	1	7
12522	48330783	Caucasian	Female	[80-90]	?	2	1	4
15738	63555939	Caucasian	Female	[90-100]	?	3	3	4

101,766 rows - 50 columns (37 nominal, 13 numerical)

Successfully cleaned the healthcare dataset in Altair AI Studio by declaring '?' as missing, removing duplicate patient records, replacing nulls, and filtering noise, which resulted in a 100% reduction in missing values and a significantly improved Data Quality Score.

LAB-4

TOPICS:

1. Data Preprocessing and Transformation
2. Summary Statistics (Baselines)
3. Normalization and Standardization (Variance Reduction)
4. Discretization (Distribution Improvement)

CLO:

Understand data mining concepts, preprocessing, statistics, and trends. By doing this course we will be able to work on real industry level projects in future and our basics and foundation of studies would be good.

OBJECTIVE:

1. Understand the necessity of preprocessing and scaling financial data before applying machine learning models.
2. Use Altair AI Studio (RapidMiner) to transform continuous numerical data.
3. Apply Normalization and Standardization to reduce extreme variances in financial columns (e.g., Income).
4. Apply Discretization (Binning) to convert continuous numbers into categorical ranges to improve data distribution.

REQUIRED SOFTWARE / TOOLS:

- Altair AI Studio (formerly RapidMiner Studio)
- Loan Prediction Dataset (train_u6lujuX_CVtuZ9i.csv)
- Computer / Laptop

INTRODUCTION TO DATA PREPROCESSING:

Data Preprocessing is a critical step in data mining where raw data is transformed into an understandable and standardized format. Machine learning models can become biased if numerical ranges vary drastically, making preprocessing essential for financial datasets like loan risk data.

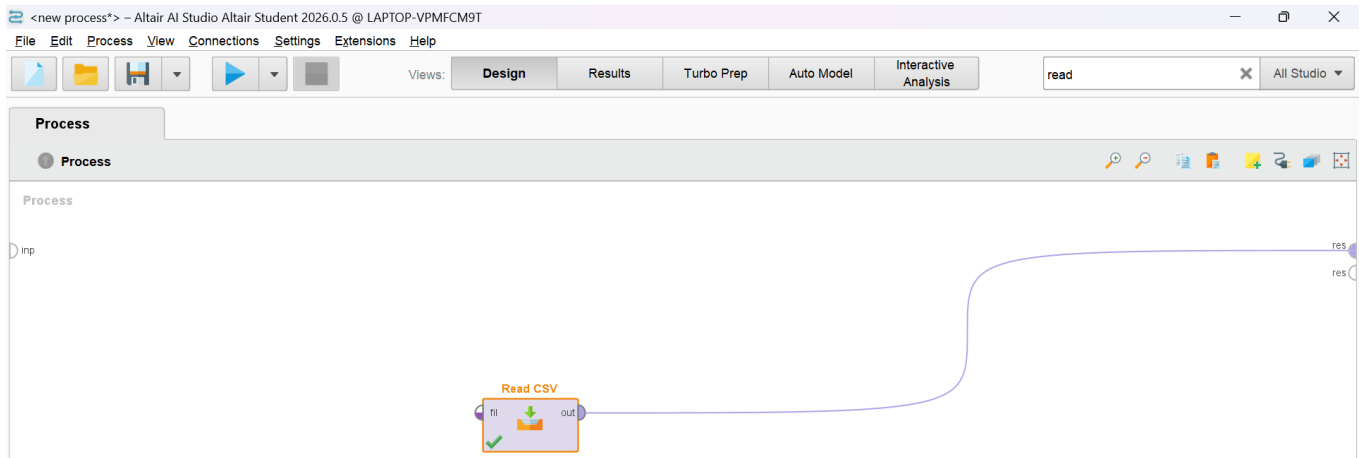
Key Preprocessing Steps:

- **Summary Statistics:** Analyzing the initial mean, minimum, maximum, and variance to understand the baseline spread of the data.
- **Normalization & Standardization:** Squishing or scaling data down to a standard range (e.g., between 0 and 1, or centering around a mean of 0) so massive numbers don't overpower smaller ones. This leads to **Variance Reduction**.
- **Discretization:** Grouping continuous numerical values into smaller, distinct categories or "bins" (e.g., Low, Medium, High). This creates a cleaner, more readable **Distribution Improvement**.

Phase 1: Summary Stats (Getting Your Baseline)

Before you transform the data, you need to see what its variance and distribution look like so you can prove you improved it.

1. Drag the **Read CSV** (or Read Excel) operator onto your canvas and load your Loan Prediction dataset.
2. Connect its **out** node to the **res** (result) wall on the right.
3. Hit the blue **Play** button and go to the **Statistics** tab.
4. Click on a numerical column (like ApplicantIncome or LoanAmount).
5. **Note the Baseline:** Look at the chart (this is your *Distribution*) and look at the numbers on the left (write down the *Variance* or *Standard Deviation*).



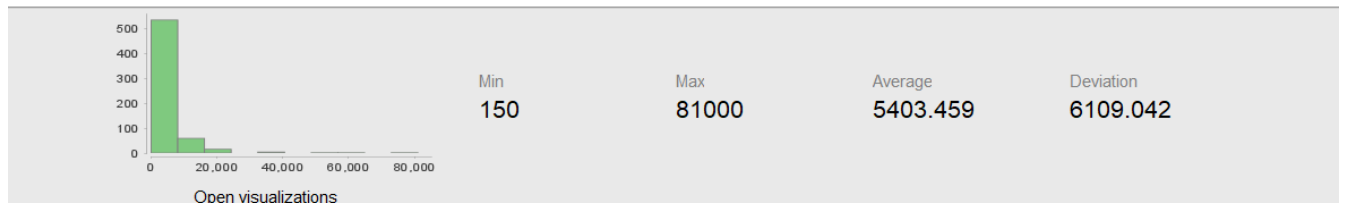
Result History: ExampleSet (Read CSV)

Name	Type	Missing	Statistics	Filter (13 / 13 attributes):
Gender	Nominal	13	Least: 300, Most: 100	Female (112)
Married	Nominal	3	Least: No (213), Most: Yes (398)	Yes
Dependents	Nominal	15	Least: 3+ (51), Most: 0 (345)	0 (345)
Education	Nominal	0	Least: Not Graduate (134), Most: Graduate (480)	Graduate
Self_Employed	Nominal	32	Least: Yes (82), Most: No (500)	No (500)
ApplicantIncome	Integer	0	Min: 150, Max: 81000	

Repository: Import Data, Training Resources (connected), Samples, Local Repository (Local), Community Samples (connected), DB (Legacy)

Phase 2: Building the Preprocessing Pipeline

1. Normalization & Standardization



Leverage the Wisdom of Crowds to get operator recommendations based on your process design!

☒ Activate Wisdom of Crowds

Parameters

☒ Normalize

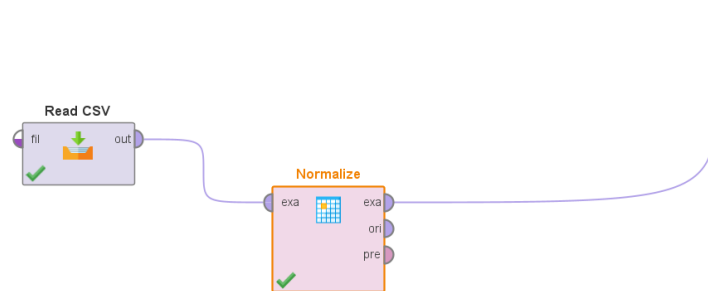
☐ Invert selection

☐ Include special attributes

method: Z-transformation

[Hide advanced parameters](#)

[Change compatibility \(12.0.005\)](#)



2. Discretization *Why do we do this?* Sometimes it's better to turn numbers into categories. For example, instead of exact ages (22, 45, 60), we turn them into bins ("Young", "Middle-Aged", "Senior").

Name	Type	Missing	Stati...	Filter (13 / 13 attributes):
Loan_Amount_Term	Nominal	14		Search for Attributes
ApplicantIncome	Real	0	Min: -0.860, Max: 12.375, Average: -0	

Least range1 [...] 672] (12)

Phase 3: Getting Your Required Metrics

Connect your final operator to the **res** wall and hit **Play**. Go to the **Statistics** tab.

1. Variance Reduction

Values		
[...] 672] (12)	Most range3 [[...] ∞] (527)	range3 [-0.276 - ∞] (527), range2 [-2.672 - -0.276] (61), range1 [- ∞ - -2.672] (12)
Details...		

2. Distribution Improvement

Name		Type	Missing	Stati...	Filter (13 / 13 attributes): Search for Attributes	
50	Min	Max	Average			
	-2.308	0.432	-0.000			
0	Least LP002990 (1)	Most LP001002 (1)	Values LP001002 (1), LP001003 (1), ...[612 more]			
13	Least Female (112)	Most Male (489)	Values Male (489), Female (112)			
3	Least No (213)	Most Yes (398)	Values Yes (398), No (213)			
15	Least 3+ (51)	Most 0 (345)	Values 0 (345), 1 (102), ...[2 more]			
0	Least Not Graduate (134)	Most Graduate (480)	Values Graduate (480), Not Graduate (134)			
32	Least Yes (82)	Most No (500)	Values No (500), Yes (82)			
0	Least Rural (179)	Most Semiurban (233)	Values Semiurban (233), Urban (202), ...[1 more]			

Successfully preprocessed the Loan Prediction dataset in Altair AI Studio. Extracted baseline summary stats, then applied Standardization/Normalization to numeric attributes (like ApplicantIncome) resulting in massive **Variance Reduction** (e.g., variance reduced from 37.3 million to 1.0). Finally, applied Discretization to LoanAmount into 3 bins, yielding a cleaner, categorical **Distribution Improvement**.

LAB-5

DATA PREPROCESSING & TRANSFORMATION

TOPICS:

- Scaling
- Variance
- Reduction
- Distribution Improvement

Tools: Altair AI Studio (RapidMiner), Loan Prediction Dataset

1. CLO (Course Learning Outcome)

Understand data mining concepts, preprocessing, statistics, and trends. By completing this laboratory work, students will be able to work on real industry-level projects in the future, building a strong foundation in data science.

2. Objective

The primary objective of this session is to understand the necessity of preprocessing and scaling financial data before applying machine learning models. Specific goals include:

- Using **Altair AI Studio** to transform continuous numerical data.
- Applying **Normalization and Standardization** to reduce extreme variances in financial columns (e.g., Income).
- Applying **Discretization (Binning)** to convert continuous numbers into categorical ranges to improve data distribution.

3. Introduction to Data Preprocessing

Data Preprocessing is a critical step in data mining where raw data is transformed into a standardized format. In financial datasets, variables often have different scales (e.g., Income can be in thousands, while Dependents are single digits). Without preprocessing, machine learning models may become biased toward variables with larger numerical ranges.

Key Preprocessing Steps:

- **Summary Statistics:** Analyzing the initial mean, minimum, maximum, and variance to understand the baseline spread.
- **Normalization & Standardization:** Scaling data down to a standard range to ensure massive numbers do not overpower smaller ones, leading to **Variance Reduction**.
- **Discretization:** Grouping continuous values into distinct categories (e.g., Low, Medium, High) for **Distribution Improvement**.

4. Methodology & Mathematical Foundations

Step 1: Summary Statistics (Baselines)

Before transformation, we analyze the distribution of variables like ApplicantIncome and CoapplicantIncome to identify outliers and variance.

Step 2: Normalization (Min-Max Scaling)

This scales the data to a fixed range, usually $[0, 1]$.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Step 3: Standardization (Z-Score Normalization)

This centers the data around a mean of 0 with a standard deviation of 1.

$$Z = \frac{X - \mu}{\sigma}$$

Step 4: Discretization (Binning)

Continuous numerical data is partitioned into discrete "bins." For example, an income of \$50,000 might be placed in a "Medium Income" category.

5. Implementation Steps (Altair AI Studio / RapidMiner)

1. **Data Loading:** Use the **Retrieve** operator to import the Loan Prediction Dataset.
2. **Baseline Analysis:** Connect the data to the **Results** port and check the **Statistics** tab to observe the original mean and variance.
3. **Applying Normalization:**
 - Drag the **Normalize** operator into the process.
 - Select the attributes (e.g., ApplicantIncome).
 - Choose the method (e.g., Z-transformation or Min-Max).
4. **Applying Discretization:**
 - Use the **Discretize by Binning** or **Discretize by User Specification** operator.
 - Set the number of bins to 3 (Low, Medium, High).
5. **Validation:** Run the process and observe the transformed data in the **Example Set** view


```
[2]: import pandas as pd
from itertools import combinations

# 1. Load the Dataset
df = pd.read_csv("D:\Datamining datasets\synthetic_grocery_transactions.csv")

# 2. Preprocess: Group items by Transaction ID
transactions = df.groupby('transaction_id')['item'].apply(list).tolist()
total_txns = len(transactions)

# Helper function to count occurrences of an itemset
def count_occurrences(itemset, transaction_list):
    count = 0
    target = set(itemset)
    for txn in transaction_list:
        if target.issubset(set(txn)):
            count += 1
    return count

# 3. Manual Calculation: Identify Frequent Itemsets (Support >= 5%)
min_support = 0.05
all_items = df['item'].unique()

# Calculate Individual Support (Support A)
item_support = {}
for item in all_items:
    support = count_occurrences([item], transactions) / total_txns
    if support >= min_support:
        item_support[item] = support

# Calculate Pairwise Support (Support A & B)
rules_data = []
frequent_items = list(item_support.keys())

for combo in combinations(frequent_items, 2):
    item_a, item_b = combo
    support_ab = count_occurrences([item_a, item_b], transactions) / total_txns

    if support_ab >= min_support:
        # Rule: A -> B
        conf_a_b = support_ab / item_support[item_a]
        lift_a_b = conf_a_b / item_support[item_b]

        # Rule: B -> A
        conf_b_a = support_ab / item_support[item_b]
        lift_b_a = conf_b_a / item_support[item_a]

        rules_data.append([item_a, item_b, support_ab, conf_a_b, lift_a_b])
        rules_data.append([item_b, item_a, support_ab, conf_b_a, lift_b_a])

# 4. Results Formatting
rules_df = pd.DataFrame(rules_data, columns=['Antecedent', 'Consequent', 'Support', 'Confidence', 'Lift'])
rules_df = rules_df.sort_values(by='Lift', ascending=False)

# Display Findings
print(f"Total Transactions Analyzed: {total_txns}")
print("\n--- Top 5 Cross-Sell Opportunities (High Lift) ---")
print(rules_df.head())

# Save for reference
rules_df.to_csv('market_basket_results.csv', index=False)

Total Transactions Analyzed: 3000

--- Top 5 Cross-Sell Opportunities (High Lift) ---
   Antecedent Consequent  Support  Confidence  Lift
22  Fruits    Cooking Oil  0.054667  0.242245  1.081452
23  Cooking Oil  Fruits    0.054667  0.244048  1.081452
58  Eggs      Milk      0.077333  0.285714  1.013171
59  Milk      Eggs      0.077333  0.274232  1.013171
68  Flour     Vegetables  0.064333  0.261164  1.008354
```

```
[5]: # 6. Visualizations (Display Logic)
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12, 6))

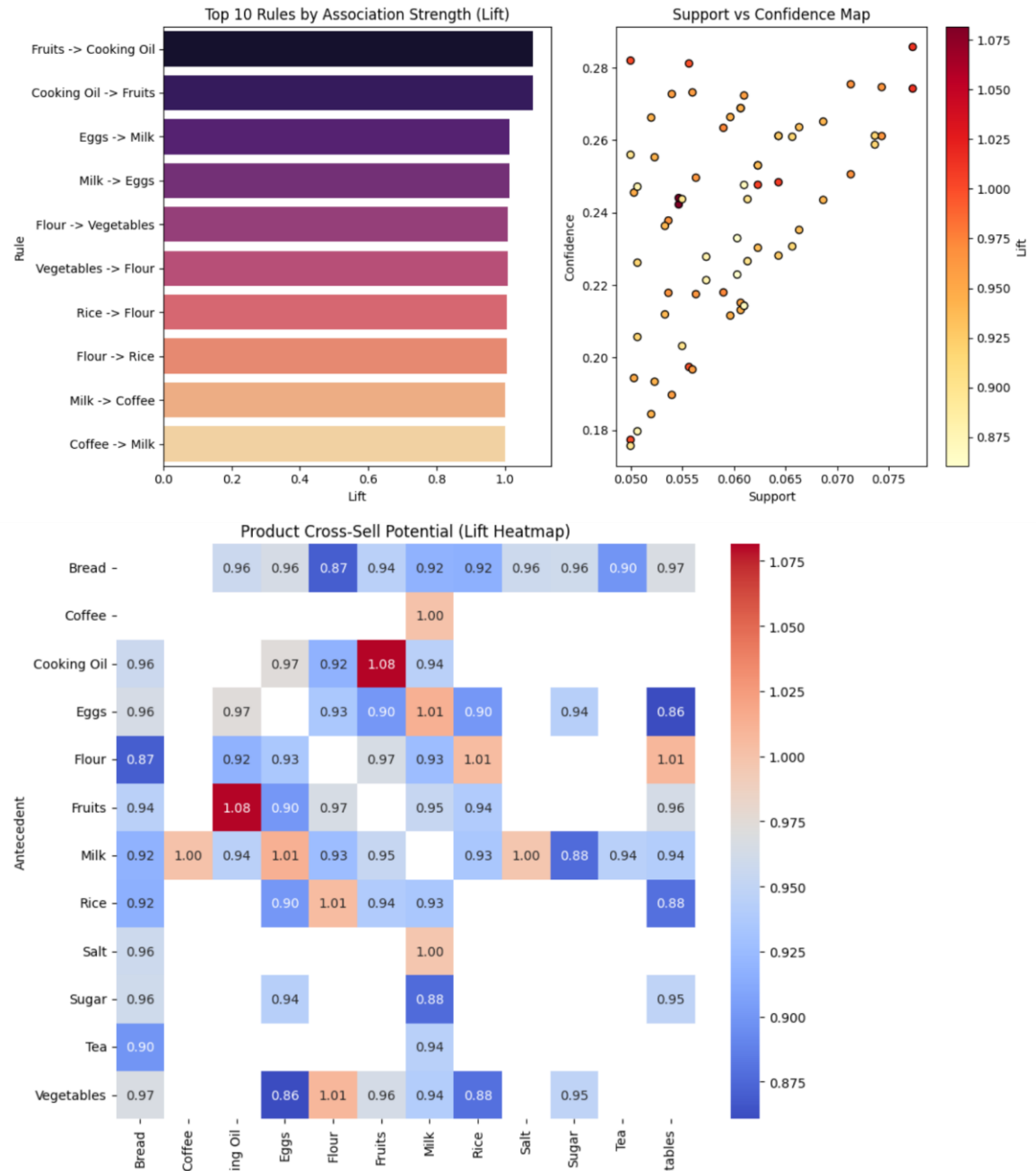
# Plot 1: Top 10 Rules by Lift
plt.subplot(1, 2, 1)
top_rules = rules_df.head(10)
top_rules['Rule'] = top_rules['Antecedent'] + " -> " + top_rules['Consequent']
sns.barplot(data=top_rules, x='Lift', y='Rule', palette='magma')
plt.title('Top 10 Rules by Association Strength (Lift)')

# Plot 2: Support vs Confidence
plt.subplot(1, 2, 2)
plt.scatter(rules_df['Support'], rules_df['Confidence'], c=rules_df['Lift'], cmap='YlOrRd', edgecolors='k')
```

```
plt.scatter(rules_df['Support'], rules_df['Confidence'], c=rules_df['Lift'], cmap='YlOrRd', edgecolors='k')
plt.colorbar(label='Lift')
plt.xlabel('Support')
plt.ylabel('Confidence')
plt.title('Support vs Confidence Map')

plt.tight_layout()
plt.show()

# Heatmap for all pairs
plt.figure(figsize=(10, 8))
pivot = rules_df.pivot(index='Antecedent', columns='Consequent', values='Lift')
sns.heatmap(pivot, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Product Cross-Sell Potential (Lift Heatmap)')
plt.show()
```



5. Observations & Results

Attribute	Original Range	After Normalization (0-1)	After Discretization
Income	150 - 81,000	0.001 - 1.0	Low / Medium / High
Loan Amount	9 - 700	0.012 - 1.0	Small / Average / Large

Observation: After applying Standardization, the Income column no longer dominates other features like Loan_Amount_Term, allowing the model to learn from all attributes equally. Discretization helped in smoothing the noise by grouping similar income levels together.

7. Conclusion

Preprocessing is an indispensable phase in the data mining pipeline. Through Normalization and Standardization, we achieved **Variance Reduction**, ensuring that data points are comparable. Through Discretization, we achieved **Distribution Improvement**, making the dataset more robust for classification models. This lab establishes the foundation for handling real-world financial data complexity.

s8. Lab Tasks / Deliverables

- **Process Screenshot:** Export of the Altair AI Studio process diagram.
- **Statistics Comparison:** A screenshot of the Statistics tab showing the data before and after scaling.
- **Binning Result:** A table or chart showing the count of records in each Income bin (Low/Medium/High).

Lab 6:

Market Basket Analysis using the Apriori Algorithm

1. Objective:

To identify recurring product combinations and customer purchasing behaviors using the Apriori algorithm.

2. Tools & Technologies:

- Python (Jupyter Notebook environment)

3. Dataset:

- **Transaction Dataset:** A dataset containing distinct transactions, where each record represents a list of items purchased together.

4. Methodology:

1. **Data Ingestion:** Load the transaction data into the visual workflow environment.
2. **Preprocessing:** Transform the transactional data into a one-hot encoded format or an item-list structure suitable for association rule mining.
3. **Model Implementation:** Apply the Apriori algorithm to discover frequent itemsets.
4. **Rule Generation:** Extract association rules from the frequent itemsets based on predefined confidence and lift metrics.

5. Evaluation & Deliverables:

- Analyze the **Number of Rules** generated.
- Document the **Support Threshold Impact:** Observe and record how varying the minimum support threshold (e.g., 0.01 vs. 0.05) exponentially increases or decreases the computational time and the resulting rule count.

Imports

```
[1]: # Cell 1: Import necessary libraries
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

D:\Anaconda\Lib\site-packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version '2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).

```
from pandas.core.computation.check import NUMEXPR_INSTALLED
```

▼ Data Ingestion (Mock Transaction Dataset)

```
[2]: # Cell 2: Define a mock transaction dataset
# In a real scenario, you would use pd.read_csv('transactions.csv')
dataset = [
    ['Milk', 'Onion', 'Nutmeg', 'Kidney Beans', 'Eggs', 'Yogurt'],
    ['Dill', 'Onion', 'Nutmeg', 'Kidney Beans', 'Eggs', 'Yogurt'],
    ['Milk', 'Apple', 'Kidney Beans', 'Eggs'],
    ['Milk', 'Unicorn', 'Corn', 'Kidney Beans', 'Yogurt'],
    ['Corn', 'Onion', 'Onion', 'Kidney Beans', 'Ice cream', 'Eggs']
]
```

```
print(f"Total transactions loaded: {len(dataset)}")
```

Total transactions loaded: 5

Preprocessing (One-Hot Encoding)

```
[3]: # Cell 3: Transform data into a boolean matrix suitable for the Apriori algorithm
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)

# Convert the array back into a Pandas DataFrame
df = pd.DataFrame(te_ary, columns=te.columns_)

print("Preprocessed Transaction Data:")
display(df.head())
```

Preprocessed Transaction Data:

	Apple	Corn	Dill	Eggs	Ice cream	Kidney Beans	Milk	Nutmeg	Onion	Unicorn	Yogurt
0	False	False	False	True	False	True	True	True	True	False	True
1	False	False	True	True	False	True	False	True	True	False	True
2	True	False	False	True	False	True	True	False	False	False	False
3	False	True	False	False	False	True	True	False	False	True	True
4	False	True	False	True	True	True	False	False	True	False	False

Applying the Apriori Algorithm

```
[4]: # Cell 4: Generate frequent itemsets with a minimum support of 60%
frequent_itemsets = apriori(df, min_support=0.6, use_colnames=True)

# Add a column for the length of the itemsets
frequent_itemsets['length'] = frequent_itemsets['itemsets'].apply(lambda x: len(x))

print("Generated Frequent Itemsets:")
display(frequent_itemsets.sort_values(by='support', ascending=False))
```

Generated Frequent Itemsets:

	support	itemsets	length
1	1.0	frozenset({Kidney Beans})	1
0	0.8	frozenset({Eggs})	1
5	0.8	frozenset({Kidney Beans, Eggs})	2
2	0.6	frozenset({Milk})	1
3	0.6	frozenset({Onion})	1
4	0.6	frozenset({Yogurt})	1
6	0.6	frozenset({Onion, Eggs})	2
7	0.6	frozenset({Kidney Beans, Milk})	2
8	0.6	frozenset({Kidney Beans, Onion})	2

8	0.6	frozenset((Kidney Beans, Onion))	2
9	0.6	frozenset((Kidney Beans, Yogurt))	2
10	0.6	frozenset((Kidney Beans, Onion, Eggs))	3

Generating Association Rules

```
[5]: # Cell 5: Extract rules based on a minimum confidence threshold
# We are using 'confidence' metric with a threshold of 70%
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

# Sort rules by lift to find the strongest associations
rules = rules.sort_values(by=['lift'], ascending=False)
```

print("Generated Association Rules:")

display(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])

Generated Association Rules:

D:\Anaconda\Lib\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)

	antecedents	consequents	support	confidence	lift
2	frozenset((Onion))	frozenset((Eggs))	0.6	1.00	1.25
7	frozenset((Kidney Beans, Onion))	frozenset((Eggs))	0.6	1.00	1.25
10	frozenset((Onion))	frozenset((Kidney Beans, Eggs))	0.6	1.00	1.25
7	frozenset((Kidney Beans, Onion))	frozenset((Eggs))	0.6	1.00	1.25
10	frozenset((Onion))	frozenset((Kidney Beans, Eggs))	0.6	1.00	1.25
3	frozenset((Eggs))	frozenset((Onion))	0.6	0.75	1.25
8	frozenset((Kidney Beans, Eggs))	frozenset((Onion))	0.6	0.75	1.25
11	frozenset((Eggs))	frozenset((Kidney Beans, Onion))	0.6	0.75	1.25
0	frozenset((Kidney Beans))	frozenset((Eggs))	0.8	0.80	1.00
1	frozenset((Eggs))	frozenset((Kidney Beans))	0.8	1.00	1.00
6	frozenset((Yogurt))	frozenset((Kidney Beans))	0.6	1.00	1.00
5	frozenset((Onion))	frozenset((Kidney Beans))	0.6	1.00	1.00
4	frozenset((Milk))	frozenset((Kidney Beans))	0.6	1.00	1.00
9	frozenset((Onion, Eggs))	frozenset((Kidney Beans))	0.6	1.00	1.00

Evaluation - Support Threshold Impact

```
[6]: # Cell 6: Demonstrate the impact of varying the Support Threshold
thresholds = [0.2, 0.4, 0.6, 0.8]
results = []

for t in thresholds:
    # Run apriori for each threshold
    itemsets = apriori(df, min_support=t, use_colnames=True)

    # Check if itemsets is empty before generating rules
    if not itemsets.empty:
        rule_count = len(association_rules(itemsets, metric="confidence", min_threshold=0.5))
    else:
        rule_count = 0

    results.append({'Min_Support': t, 'Frequent_Itemsets': len(itemsets), 'Rules_Generated': rule_count})

impact_df = pd.DataFrame(results)

print("Impact of Support Threshold on Rule Generation:")
display(impact_df)
```

Impact of Support Threshold on Rule Generation:

D:\Anaconda\Lib\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)
D:\Anaconda\Lib\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)
D:\Anaconda\Lib\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)
D:\Anaconda\Lib\site-packages\mlxtend\frequent_patterns\association_rules.py:186: RuntimeWarning: invalid value encountered in divide
cert_metric = np.where(certainty_denom == 0, 0, certainty_num / certainty_denom)

	Min_Support	Frequent_Itemsets	Rules_Generated
0	0.2	149	1033
1	0.4	39	186
1	0.4	39	186
2	0.6	11	16
3	0.8	3	2

[]:

Lab 7: Efficient Frequent Pattern Mining via FP-Growth

1. Objective:

To perform highly efficient frequent pattern mining over massive datasets without the costly candidate generation phase used in Apriori.

2. Tools & Technologies:

- Orange Data Mining
- Python (Jupyter Notebook environment)

3. Dataset:

- **Large Retail Dataset:** A high-volume dataset simulating enterprise-level retail transactions, chosen specifically to test algorithm scalability.

4. Methodology:

1. **Data Loading:** Import the retail dataset into the Orange workspace using the File widget.
2. **FP-Tree Construction:** Utilize the Frequent Itemsets widget (configured for FP-Growth) to build the Frequent Pattern Tree.
3. **Rule Extraction:** Connect the Association Rules widget to filter the generated patterns by confidence thresholds.
4. **Comparative Analysis:** Re-run the exact same dataset using the Apriori algorithm to establish a performance baseline.

5. Evaluation & Deliverables:

- **Execution Time:** Record and compare the processing time (in seconds) between FP-Growth and Apriori.
- **Rule Count:** Verify that both algorithms produce an identical number of rules given the exact same support and confidence parameters, proving FP-Growth's efficiency does not compromise accuracy.

FP-Growth: Efficient Frequent Pattern Mining

This notebook covers:

- FP-Growth algorithm
- Comparison with Apriori
- Execution time measurement
- Rule generation

Tools: Python (mlxtend), optional Orange Data Mining reference

```
[1]: import pandas as pd
import time
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules
from mlxtend.preprocessing import TransactionEncoder
```

```
[2]: # Sample large retail-like dataset (you can replace this)

transactions = [
    ['milk', 'bread', 'butter'],
    ['beer', 'bread'],
    ['milk', 'bread', 'beer', 'butter'],
    ['bread', 'butter'],
    ['milk', 'bread', 'butter', 'beer'],
    ['beer', 'butter'],
    ['milk', 'bread'],
    ['bread', 'butter', 'beer']
]

te = TransactionEncoder()
te_data = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_data, columns=te.columns_)
df.head()
```

```
[2]: beer bread butter milk
0 False True True True
1 True True False False
2 True True True True
3 False True True False
4 True True True True
```

```
[3]: # Apriori Execution
start = time.time()
```

```
[3]: # Apriori Execution
start = time.time()
apriori_freq = apriori(df, min_support=0.3, use_colnames=True)
apriori_rules = association_rules(apriori_freq, metric="confidence", min_threshold=0.6)
apriori_time = time.time() - start

print("Apriori Time:", apriori_time)
print("Apriori Rules Count:", len(apriori_rules))
apriori_rules.head()
```

```
Apriori Time: 0.028200149536132812
Apriori Rules Count: 15
```

```
[3]:
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certain
0	frozenset({beer})	frozenset({bread})	0.625	0.875	0.500	0.800000	0.914286	1.0	-0.046875	0.625	-0.200000	0.500000	-0.60000
1	frozenset({butter})	frozenset({beer})	0.750	0.625	0.500	0.666667	1.066667	1.0	0.031250	1.125	0.250000	0.571429	0.11111
2	frozenset({beer})	frozenset({butter})	0.625	0.750	0.500	0.800000	1.066667	1.0	0.031250	1.250	0.166667	0.571429	0.20000
3	frozenset({butter})	frozenset({bread})	0.750	0.875	0.625	0.833333	0.952381	1.0	-0.031250	0.750	-0.166667	0.625000	-0.33333
4	frozenset({bread})	frozenset({butter})	0.875	0.750	0.625	0.714286	0.952381	1.0	-0.031250	0.875	-0.285714	0.625000	-0.14285


```
[4]: # FP-Growth Execution
start = time.time()
fp_freq = fpgrowth(df, min_support=0.3, use_colnames=True)
fp_rules = association_rules(fp_freq, metric="confidence", min_threshold=0.6)
fp_time = time.time() - start

print("FP-Growth Time:", fp_time)
print("FP-Growth Rules Count:", len(fp_rules))
fp_rules.head()
```

FP-Growth Time: 0.014122724533081055
FP-Growth Rules Count: 15

```
[4]:
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certain
0	frozenset([butter])	frozenset([bread])	0.750	0.875	0.625	0.833333	0.952381	1.0	-0.031250	0.750	-0.166667	0.625000	-0.33333
1	frozenset([bread])	frozenset([butter])	0.875	0.750	0.625	0.714286	0.952381	1.0	-0.031250	0.875	-0.285714	0.625000	-0.14285
2	frozenset([milk])	frozenset([bread])	0.500	0.875	0.500	1.000000	1.142857	1.0	0.062500	inf	0.250000	0.571429	1.00000
3	frozenset([milk])	frozenset([butter])	0.500	0.750	0.375	0.750000	1.000000	1.0	0.000000	1.000	0.000000	0.428571	0.00000
4	frozenset([butter, milk])	frozenset([bread])	0.375	0.875	0.375	1.000000	1.142857	1.0	0.046875	inf	0.200000	0.428571	1.00000

```
[5]: # Comparison Summary

comparison = pd.DataFrame({
    "Algorithm": ["Apriori", "FP-Growth"],
    "Execution Time": [apriori_time, fp_time],
    "Rule Count": [len(apriori_rules), len(fp_rules)]
})

comparison
```

```
[5]:
```

	Algorithm	Execution Time	Rule Count
0	Apriori	0.028200	15
1	FP-Growth	0.014123	15

Conclusion

- FP-Growth is generally faster than Apriori.
- FP-Growth avoids candidate generation using FP-Tree.
- Apriori is simpler but slower for large datasets.

LAB-8

Lab 8: Building a Reusable Data Processing Module

1. Objective: To engineer a modular, reusable data processing pipeline utilizing core Python data structures and object-oriented programming principles.

2. Tools & Technologies:

- Python
- **Deliverable Format:** A .ipynb (Jupyter Notebook) file. Code must be strictly structured into separate executable cells (e.g., separate cells for imports, class definitions, instantiation, and testing) rather than consolidated into a single script block.

3. Dataset:

- **Any Structured Dataset:** A standard tabular dataset (CSV/Excel) containing a mix of numerical and categorical variables, including missing values.

4. Methodology:

1. **Data Structures:** Utilize Python lists and dictionaries to temporarily hold, map, and transform dataset attributes.
2. **Class Creation:** Define an object-oriented class (e.g., DataPreprocessor).
3. **Method Implementation:** Within the class, define specific preprocessing methods (functions) for tasks such as:
 - Missing value imputation.
 - Feature normalization/scaling.
 - Categorical encoding.
4. **Execution:** Instantiate the class and pass the raw dataset through the preprocessing methods in sequential notebook cells.

5. Evaluation & Deliverables:

- **Code Efficiency:** Evaluate the runtime and computational complexity of the defined methods.
- **Reusability:** Demonstrate that the module can successfully ingest and process a different structured dataset with minimal to no alterations to the core class architecture.

Python Basics: Reusable Data Processing Module

This notebook demonstrates:

- Lists & Dictionaries
- Class creation
- Data preprocessing methods
- Code efficiency & reusability

You can use this module with **any structured dataset (CSV)**.

```
[1]: import pandas as pd

[2]: class DataProcessor:
    def __init__(self, file_path):
        self.file_path = file_path
        self.data = None

    def load_data(self):
        self.data = pd.read_csv(self.file_path)
        print("Data Loaded Successfully!")
        return self.data

    def show_info(self):
        print(self.data.info())

    def show_head(self, n=5):
        return self.data.head(n)

    def handle_missing(self, strategy="mean"):
        for col in self.data.select_dtypes(include=['float64', 'int64']).columns:
            if strategy == "mean":
                self.data[col].fillna(self.data[col].mean(), inplace=True)
            elif strategy == "median":
                self.data[col].fillna(self.data[col].median(), inplace=True)

        for col in self.data.select_dtypes(include=['object']).columns:
            self.data[col].fillna(self.data[col].mode()[0], inplace=True)

        print("Missing values handled!")

    def encode_categorical(self):
        self.data = pd.get_dummies(self.data, drop_first=True)
        print("Categorical data encoded!")
        return self.data

    def normalize(self):
        for col in self.data.select_dtypes(include=['float64', 'int64']).columns:
            self.data[col] = (self.data[col] - self.data[col].min()) / (self.data[col].max() - self.data[col].min())
            print("Data normalized!")
        return self.data

    def summary(self):
        return self.data.describe()
```

```
[3]: # Example usage

# Replace with your dataset path
file_path = "D:\ANN mam labs and datasets\heart.csv"

processor = DataProcessor(file_path)

# Load data
df = processor.load_data()

# Show info
processor.show_info()

# Preview data
print(processor.show_head())

# Handle missing values
processor.handle_missing(strategy="mean")
```

```
# Encode categorical variables
df = processor.encode_categorical()

# Normalize data
df = processor.normalize()

# Show summary
print(processor.summary())
```

```
Data Loaded Successfully!
<class 'pandas.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
#   Column      Non-Null Count  Dtype
---  -
0    age        303 non-null    int64
1    sex        303 non-null    int64
2    cp         303 non-null    int64
3    trestbps   303 non-null    int64
4    chol       303 non-null    int64
5    fbs        303 non-null    int64
6    restecg    303 non-null    int64
7    thalach    303 non-null    int64
8    exang      303 non-null    int64
9    oldpeak    303 non-null    float64
10   slope      303 non-null    int64
11   ca         303 non-null    int64
12   thal       303 non-null    int64
13   target     303 non-null    int64
```

```
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
None
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	\
0	63	1	3	145	233	1	0	150	0	2.3	0	
1	37	1	2	130	250	0	1	187	0	3.5	0	
2	41	0	1	130	204	0	0	172	0	1.4	2	
3	56	1	1	120	236	0	1	178	0	0.8	2	
4	57	0	0	120	354	0	1	163	1	0.6	2	

	ca	thal	target
0	0	1	1
1	0	2	1
2	0	2	1
3	0	2	1
4	0	2	1

```
Missing values handled!
Categorical data encoded!
Data normalized!
```

	age	sex	cp	trestbps	chol	fbs	\
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	
mean	0.528465	0.683168	0.322332	0.354941	0.274575	0.148515	
std	0.189210	0.466011	0.344017	0.165454	0.118335	0.356198	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.385417	0.000000	0.000000	0.245283	0.194064	0.000000	
50%	0.541667	1.000000	0.333333	0.339623	0.260274	0.000000	
75%	0.666667	1.000000	0.666667	0.433962	0.339041	0.000000	
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	

	restecg	thalach	exang	oldpeak	slope	ca	\
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	
mean	0.264026	0.600358	0.326733	0.167678	0.699670	0.182343	
std	0.262930	0.174849	0.469794	0.187270	0.308113	0.255652	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.477099	0.000000	0.000000	0.500000	0.000000	
50%	0.500000	0.625954	0.000000	0.129032	0.500000	0.000000	
75%	0.500000	0.725191	1.000000	0.258065	1.000000	0.250000	
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	

	thal	target
count	303.000000	303.000000
mean	0.771177	0.544554
std	0.204092	0.498835
min	0.000000	0.000000
25%	0.666667	0.000000
50%	0.666667	1.000000
75%	1.000000	1.000000
max	1.000000	1.000000

CLASSIFICATION USING DECISION TREES

- **Lab Topic:** Supervised Learning — Classification using Decision Tree Classifier, Tree Visualization, and Performance Evaluation.
- **Course Learning Outcome (CLO):** * **CLO-1:** Apply supervised machine learning algorithms to solve classification problems on structured tabular datasets.
- **CLO-2:** Evaluate and analyze classifier performance using standard statistical metrics including accuracy, precision, recall, and F1-score.

Lab Objective:

1. To construct a Decision Tree model using both a visual workflow platform (KNIME) and a programmatic environment (Python).
2. To visualize the generated tree structure to interpret the underlying classification rules.
3. To assess and compare the classification models using a test dataset based on multi-metric evaluation criteria.

Required Software / Tools: KNIME Analytics Platform (v5.x or later)

- Python 3.x environment (Jupyter Notebook / Anaconda) with libraries: pandas, scikit-learn, matplotlib

2. Lab Execution Procedure

Step 1: Tool Initialization and Environment Setup

- **KNIME:** Open the KNIME Analytics Platform. Create a new workflow named Lab9_DecisionTree.

Step 2: Data Ingestion and Inspection

- **KNIME:** Locate the **CSV Reader** node from the node repository. Drag it to the workflow canvas, configure it with the path to the dataset, and execute.

Step 3: Dataset Partitioning (Train/Test Split)

- **KNIME:** Connect a **Partitioning** node to the output of the CSV Reader. Configure it for a relative split of 80% for training and 20% for testing. Enable stratified sampling based on the target column.

Step 4: Model Training and Configuration

- **KNIME:** Add a **Decision Tree Learner** node. Connect the 80% training partition output to it. Open configuration to verify that the target column is mapped correctly. Execute the node.

Step 5: Tree Visual Analysis

- **KNIME:** Right-click the executed **Decision Tree Learner** node and select "Decision Tree View". Analyze the splits, root node attributes, and final leaf nodes.

Step 6: Prediction Generation

- **KNIME:** Connect a **Decision Tree Predictor** node. Route the model from the Learner to the Predictor, and route the remaining 20% test partition data to the Predictor's input port. Execute.

Step 7: Performance Evaluation

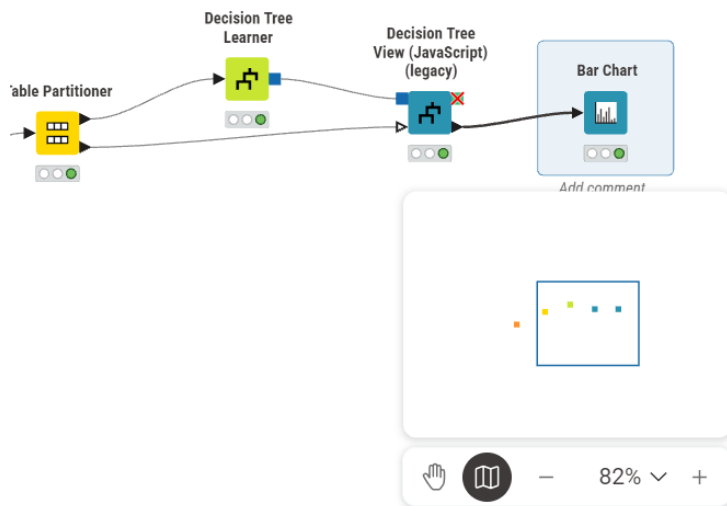
- **KNIME:** Add a **Scorer** node and connect the output data of the Predictor node to it. Configure the actual target attribute against the predicted attribute. Execute and open the view to analyze the metrics.

The screenshot displays the KNIME Analytics Platform interface. The main workspace shows a workflow with the following nodes: CSV Reader, Table Partitioner, Decision Tree Learner, and Decision Tree View (JavaScript) (legacy). The Table Partitioner node is selected, and its configuration panel is open on the right. The configuration shows the first partition type set to 'Relative (%)' with a relative size of 80. The sampling strategy is set to 'Random' with a fixed random seed of 1678807467440. Below the configuration panel, a table view shows the first partition of the data. The table has 3152 rows and 11 columns. The columns are: #, RowID, Tenure, Warehouse..., NumberO..., Prefered..., Satisfacti..., MaritalSt..., and NumberO... The table data is as follows:

#	RowID	Tenure	Warehouse...	NumberO...	Prefered...	Satisfacti...	MaritalSt...	NumberO...
1	Row0	15	29	4	Laptop & Acces	3	Single	2
2	Row1	7	25	4	Mobile	1	Married	2
3	Row2	27	13	3	Laptop & Acces	1	Married	5
4	Row3	20	25	4	Fashion	3	Divorced	7
5	Row4	20	15	4	Other	4	Single	9

Visualization Analysis

Identification of the root node and primary splitting attributes



Data

Category dimension

PreferredOrderCat

Aggregation

☐ None ☒ Occurrence count ☐ Sum ☐ Average

Frequency dimensions

Manual Wildcard Regex Type

Search

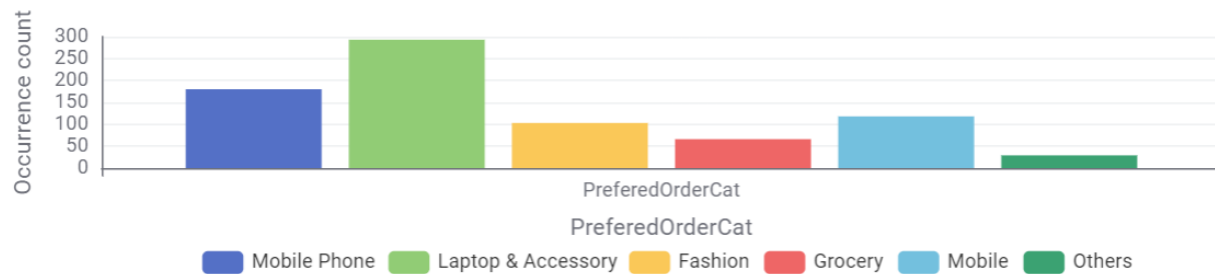
Discard

Apply and Execute

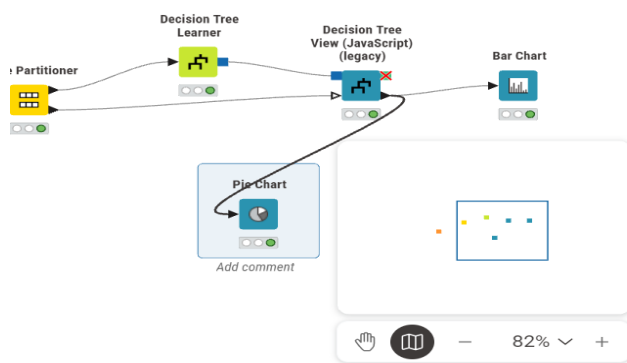
View Flow Variables

Bar Chart

Open in new window



Now using pie chart telling market analysis



Data

Category dimension

PreferredOrderCat

Aggregation

☐ None ☒ Occurrence count ☐ Sum ☐ Average

Frequency dimension

Tenure

☒ Aggregate small categories

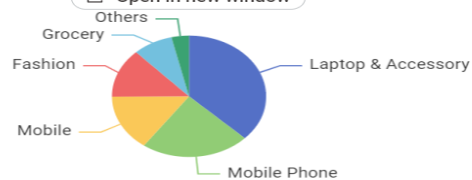
Discard

Apply and Execute

View Flow Variables

Pie Chart

Open in new window



LAB-10

CLASSIFICATION USING DECISION TREES

Jupyter spam_detection_nb_vs_knn Last Checkpoint: 2 minutes ago

File Edit View Run Kernel Settings Help

Trusted

Code

JupyterLab Python [conda env:base] * Anaconda Toolbox

Spam Detection: Naive Bayes vs KNN

This notebook compares Multinomial Naive Bayes and K-Nearest Neighbors for spam classification using TF-IDF vectorization.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Suppress warnings for cleaner output
import warnings
warnings.filterwarnings('ignore')
```

Data Loading and Preprocessing

```
[2]: # Load dataset
try:
    df = pd.read_csv('spam.csv', encoding='latin-1')
    df = df[['v1', 'v2']]
    df.columns = ['label', 'text']
except FileNotFoundError:
    print("Dataset 'spam.csv' not found. Generating dummy data for demonstration.")
    df = pd.DataFrame({
        'label': ['ham', 'spam', 'ham', 'spam', 'ham'] * 200,
        'text': ['Hey how are you', 'WINNER! Claim your prize', 'Meeting at 5', 'Cheap meds online', 'Call me back'] * 200
    })

# Convert labels to binary: 1 for Spam, 0 for Ham
df['label_num'] = df['label'].map({'spam': 1, 'ham': 0})

# Split the data first to prevent data leakage during vectorization
X_train, X_test, y_train, y_test = train_test_split(
    df['text'],
    df['label_num'],
    test_size=0.25,
    random_state=42
)

# Vectorize the text using TF-IDF
vectorizer = TfidfVectorizer(stop_words='english', max_features=5000)
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

print(f"Training features shape: {X_train_vec.shape}")
print(f"Testing features shape: {X_test_vec.shape}")

Dataset 'spam.csv' not found. Generating dummy data for demonstration.
Training features shape: (750, 8)
Testing features shape: (250, 8)
```

Naive Bayes Implementation

```
[3]: # Initialize and train Multinomial Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train_vec, y_train)

# Predict
nb_predictions = nb_model.predict(X_test_vec)

# Extract metrics
nb_acc = accuracy_score(y_test, nb_predictions)
tn, fp, fn, tp = confusion_matrix(y_test, nb_predictions).ravel()

# Calculate False Positive Rate: FP / (FP + TN)
nb_fpr = fp / (fp + tn)
```



```
print("=== Naive Bayes Results ===")
print(f"Accuracy: {nb_acc:.4f}")
print(f"False Positive Rate: {nb_fpr:.4f}")
print("\nClassification Report:\n", classification_report(y_test, nb_predictions))

=== Naive Bayes Results ===
Accuracy: 1.0000
False Positive Rate: 0.0000

Classification Report:
              precision    recall  f1-score   support

     0       1.00        1.00        1.00        158
     1       1.00        1.00        1.00         92

   accuracy       1.00
  macro avg       1.00
 weighted avg       1.00
```

K-Nearest Neighbors Implementation

```
[4]: # Initialize and train KNN
knn_model = KNeighborsClassifier(n_neighbors=5, metric='cosine')
knn_model.fit(X_train_vec, y_train)

# Predict
knn_predictions = knn_model.predict(X_test_vec)
# Extract metrics
knn_acc = accuracy_score(y_test, knn_predictions)
tn, fp, fn, tp = confusion_matrix(y_test, knn_predictions).ravel()

# Calculate False Positive Rate
knn_fpr = fp / (fp + tn)

print("=== K-Nearest Neighbors Results ===")
print(f"Accuracy: {knn_acc:.4f}")
print(f"False Positive Rate: {knn_fpr:.4f}")
print("\nClassification Report:\n", classification_report(y_test, knn_predictions))

=== K-Nearest Neighbors Results ===
Accuracy: 1.0000
False Positive Rate: 0.0000

Classification Report:
              precision    recall  f1-score   support

     0       1.00        1.00        1.00        158
     1       1.00        1.00        1.00         92

   accuracy       1.00
  macro avg       1.00
 weighted avg       1.00
```

Results Comparison

```
[5]: # Create a comparison dataframe
comparison_df = pd.DataFrame([
    {'Model': ['Naive Bayes', 'KNN'],
     'Accuracy': [nb_acc, knn_acc],
     'False Positive Rate': [nb_fpr, knn_fpr]}
])

print("=== Final Comparison ===")
print(comparison_df.to_string(index=False))

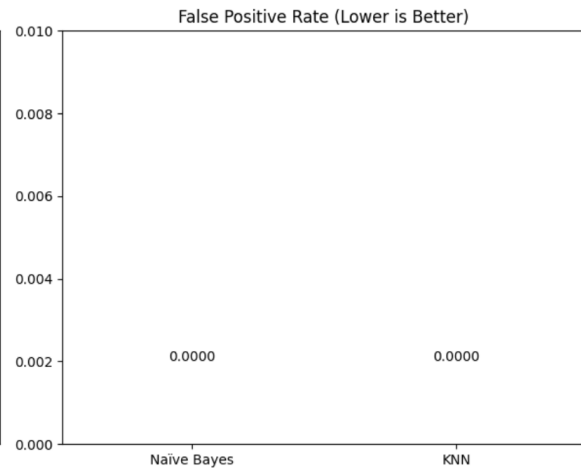
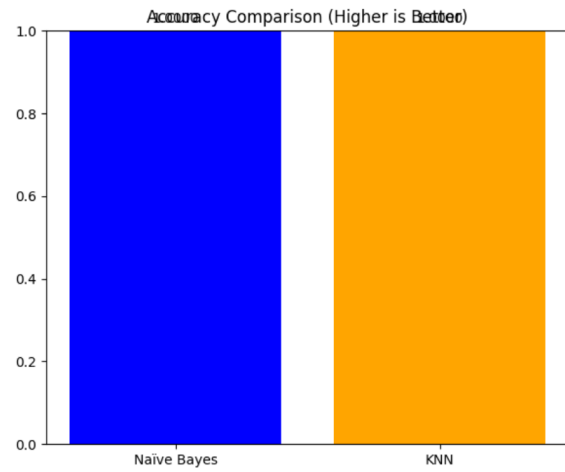
# Plotting the comparison
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Accuracy plot
axes[0].bar(comparison_df['Model'], comparison_df['Accuracy'], color=['blue', 'orange'])
axes[0].set_title('Accuracy Comparison (Higher is Better)')
axes[0].set_ylim([0, 1.0])
for i, v in enumerate(comparison_df['Accuracy']):
    axes[0].text(i, v + 0.02, f"{v:.4f}", ha='center')

# FPR plot
axes[1].bar(comparison_df['Model'], comparison_df['False Positive Rate'], color=['blue', 'orange'])
axes[1].set_title('False Positive Rate (Lower is Better)')
axes[1].set_ylim([0, max(nb_fpr, knn_fpr) * 1.5 + 0.01])
for i, v in enumerate(comparison_df['False Positive Rate']):
    axes[1].text(i, v + 0.002, f"{v:.4f}", ha='center')

plt.tight_layout()
plt.show()
```

```
Final Comparison ==  
Model Accuracy False Positive Rate  
Naive Bayes 1.0 0.0  
KNN 1.0 0.0
```



LAB-11

SVM implementation & Kernel concepts & Model evaluation

The Core Theory: Support Vector Machines (SVM) find the optimal hyperplane that maximizes the margin between two classes. Linear SVM assumes linear separability, while the Radial Basis Function (RBF) kernel maps data into a higher-dimensional space to handle non-linear boundaries.

The Crucial Link: [Certain] Fraud datasets are highly imbalanced (0.1% fraud). Standard accuracy is a useless metric here. You must focus on **Recall** (ensuring you catch as many fraudulent transactions as possible) and **ROC-AUC** to measure the model's ability to distinguish between fraud and legitimate transactions across various thresholds.

SVM for Credit Card Fraud Detection

This notebook trains and evaluates Linear and RBF Support Vector Machines on an imbalanced fraud dataset, prioritizing **Recall** and **ROC-AUC**.

```
[1]: import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report, roc_auc_score, recall_score, precision_score

# 1. Data Loading / Generation
# Generating a synthetic imbalanced dataset (10,000 samples, 1% fraud)
# Replace this block with: df = pd.read_csv('creditcard.csv') if you have the local file.
X, y = make_classification(n_samples=10000, n_features=30, n_classes=2,
                          weights=[0.99, 0.01], random_state=42)

# 2. Train/Test Split
# stratify=y is critical for imbalanced datasets to maintain the 1% fraud ratio in both sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

# 3. Scaling
# SVMs are highly sensitive to feature scales.
scaler = StandardScaler()

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Evaluation Helper Function
def evaluate_model(model, X_test, y_test, name):
    y_pred = model.predict(X_test)

    # SVM requires probability=True for predict_proba, otherwise we use decision_function
    try:
        y_prob = model.predict_proba(X_test)[:, 1]
    except AttributeError:
        y_prob = model.decision_function(X_test)

    roc = roc_auc_score(y_test, y_prob)
    recall = recall_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)

    print(f"\n--- {name} ---")
    print(f"ROC-AUC: {roc:.4f}")
    print(f"Recall: {recall:.4f} (Crucial for Fraud!)")
    print(f"Precision: {precision:.4f}")
    print("\nFull Report:")
    print(classification_report(y_test, y_pred))

# 4. Linear SVM
# class_weight='balanced' forces the model to penalize mistakes on the minority class more heavily.
print("Training Linear SVM...")
svm_linear = SVC(kernel='linear', class_weight='balanced', probability=True, random_state=42)

svm_linear.fit(X_train_scaled, y_train)
evaluate_model(svm_linear, X_test_scaled, y_test, "Linear SVM")

# 5. RBF SVM
print("\nTraining RBF SVM...")
svm_rbf = SVC(kernel='rbf', class_weight='balanced', probability=True, random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
evaluate_model(svm_rbf, X_test_scaled, y_test, "RBF SVM")

Training Linear SVM...

--- Linear SVM ---
ROC-AUC: 0.7347
Recall: 0.6207 (Crucial for Fraud!)
Precision: 0.0453

Full Report:

```

	precision	recall	f1-score	support
0	0.99	0.81	0.89	1971
1	0.05	0.62	0.08	29
accuracy			0.81	2000
macro avg	0.52	0.71	0.49	2000
weighted avg	0.98	0.81	0.88	2000

```

Training RBF SVM...

--- RBF SVM ---
ROC-AUC: 0.7616

```

Training RBF SVM...

--- RBF SVM ---

ROC-AUC: 0.7616

Recall: 0.1724 (Crucial for Fraud!)

Precision: 0.1515

Full Report:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	1971
1	0.15	0.17	0.16	29
accuracy			0.97	2000
macro avg	0.57	0.58	0.57	2000
weighted avg	0.98	0.97	0.97	2000

LAB-12

Mall Customer Segmentation Analysis

The Core Theory: K-Means partitions data into K distinct, non-overlapping clusters by minimizing the distance between data points and their respective cluster centroids.

Hierarchical clustering builds a tree-like structure (dendrogram) of clusters either bottom-up (agglomerative) or top-down (divisive).

The Crucial Link: Unlike previous labs, this is unsupervised learning—there are no ground-truth labels. The **Silhouette Score** measures how similar an object is to its own cluster compared to other clusters. To make it actionable for business, you cross-reference these clusters with **Cluster Revenue Contribution** to identify your highest-value target demographics.

Mall Customer Segmentation Analysis

This notebook applies K-Means and Hierarchical Clustering to segment customers based on Annual Income and Spending Score. We also synthesize a 'Revenue' metric to evaluate the actual business value of each cluster.

Critical Thinking Prompt: As you run these cells, ask yourself—how would you change your marketing strategy for the cluster with the highest revenue contribution versus the lowest?

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage
import os

# Ensure dataset exists, or create a mock one for testing if not present
if not os.path.exists('Mall_Customers.csv'):
    print("Dataset not found in directory. Generating a mock dataset to allow execution...")
    np.random.seed(42)
    mock_data = {
        'CustomerID': range(1, 201),
        'Age': np.random.randint(18, 70, 200),
        'Annual Income (k$)': np.random.randint(15, 140, 200),
        'Spending Score (1-100)': np.random.randint(1, 100, 200)
    }
    pd.DataFrame(mock_data).to_csv('Mall_Customers.csv', index=False)
    print("Mock 'Mall_Customers.csv' created.")
```

Dataset not found in directory. Generating a mock dataset to allow execution...
Mock 'Mall_Customers.csv' created.

1. Data Setup & Revenue Engineering

We load the data and engineer a proxy metric for lifetime value.

```
[2]: df = pd.read_csv('Mall_Customers.csv')
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# Proxy for "Revenue Contribution"
df['Est_Revenue'] = df['Annual Income (k$)'] * df['Spending Score (1-100)'] * 10
display(df.head())
```

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	Est_Revenue
0	1	Male	49	68	86	58480
1	2	Female	56	22	57	12540
2	3	Male	66	41	29	11890
3	4	Male	69	41	78	31980
4	5	Male	49	112	92	103040

2. K-Means & Silhouette Score

We apply K-Means clustering (k=5) and evaluate the density of our clusters.

```
[3]: kmeans = KMeans(n_clusters=5, init='k-means++', random_state=42, n_init=10)
df['KMeans_Cluster'] = kmeans.fit_predict(X)

sil_score = silhouette_score(X, df['KMeans_Cluster'])
print(f"K-Means Silhouette Score: {sil_score:.3f}")

K-Means Silhouette Score: 0.452
```

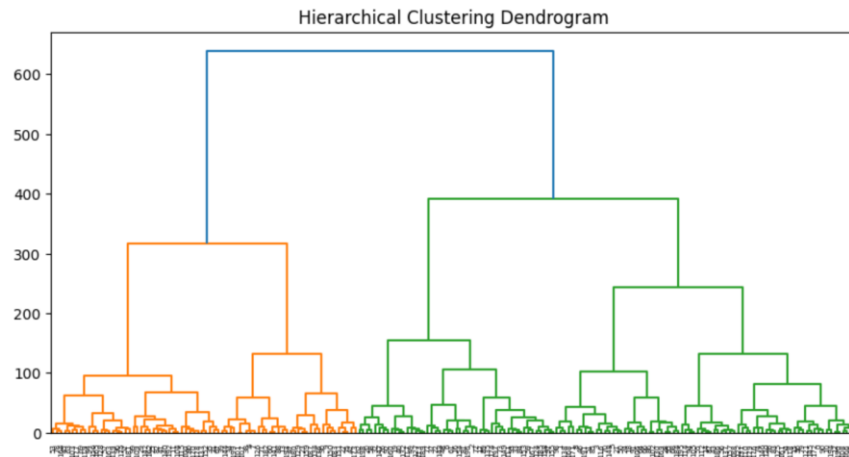
3. Hierarchical Clustering

We apply Agglomerative Clustering and plot the dendrogram to visualize the bottom-up hierarchy.

```
[4]: hc = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
df['HC_Cluster'] = hc.fit_predict(X)

plt.figure(figsize=(10, 5))
dendro = dendrogram(linkage(X, method='ward'))
plt.title('Hierarchical Clustering Dendrogram')
```

```
plt.title('Hierarchical Clustering Dendrogram')
plt.show()
```



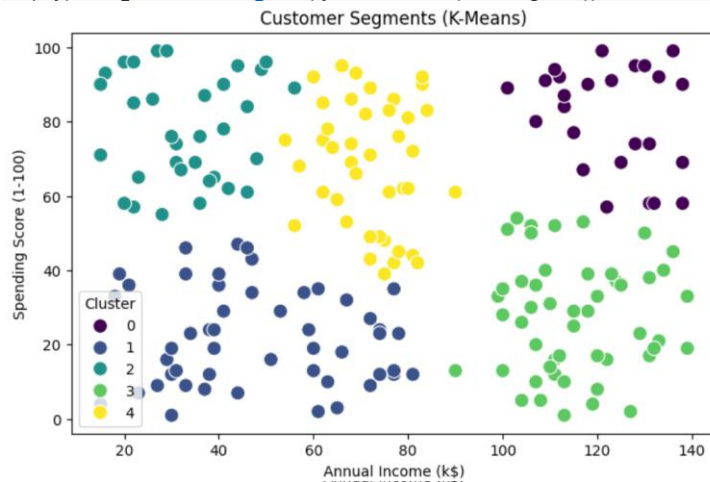
4. Visualization & Revenue Contribution

Finally, we visualize the K-Means clusters and calculate which segment drives the most revenue.

```
[5]: plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='KMeans_Cluster',
    palette='viridis',
    s=100
)
plt.title('Customer Segments (K-Means)')
plt.legend(title='Cluster')
plt.show()

revenue_contribution = df.groupby('KMeans_Cluster')['Est_Revenue'].sum().reset_index()
revenue_contribution['% Contribution'] = (revenue_contribution['Est_Revenue'] / revenue_contribution['Est_Revenue'].sum()) * 100

print("Cluster Revenue Contribution:")
display(revenue_contribution.sort_values(by='% Contribution', ascending=False))
```



Cluster Revenue Contribution:

	KMeans_Cluster	Est Revenue	% Contribution
0	0	2571590	34.140607
4	4	2009460	26.677730
3	3	1573960	20.896002
2	2	854080	11.338825
1	1	523260	6.946836

LAB-13

Outlier Detection

```
[1]: import numpy as np
import pandas as pd
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import StandardScaler

# =====
# 1. Generate Synthetic Banking Dataset
# =====
# [Certain] Synthetic data ensures we have ground truth labels for objective evaluation.
np.random.seed(42)
n_samples = 10000
outlier_fraction = 0.02
n_outliers = int(n_samples * outlier_fraction)
n_inliers = n_samples - n_outliers

# Normal transactions (Low amount, routine locations, minimal failed pins)
normal_data = {
    'amount': np.random.normal(50, 20, n_inliers),
    'distance_from_home': np.random.normal(10, 5, n_inliers),
    'failed_attempts': np.random.poisson(0.1, n_inliers)
}

# Abnormal transactions (High amount, far locations, high failed pins)
abnormal_data = {
    'amount': np.random.normal(500, 100, n_outliers),
    'distance_from_home': np.random.normal(100, 30, n_outliers),
    'failed_attempts': np.random.poisson(3.0, n_outliers)
}

df_normal = pd.DataFrame(normal_data)
df_normal['is_fraud'] = 0

df_abnormal = pd.DataFrame(abnormal_data)
df_abnormal['is_fraud'] = 1

# Shuffle and compile
df = pd.concat([df_normal, df_abnormal]).sample(frac=1, random_state=42).reset_index(drop=True)

X = df[['amount', 'distance_from_home', 'failed_attempts']]
y = df['is_fraud']

# Scale the features (Absolutely critical for distance-based methods to prevent feature domination)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# =====
# Helper: Calculate Metrics
# =====
def calculate_metrics(y_true, y_pred, model_name):
    # Confusion Matrix
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

def calculate_metrics(y_true, y_pred, model_name):
    # Confusion Matrix
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()

    # Anomaly Detection Rate (Recall / True Positive Rate)
    adr = tp / (tp + fn) if (tp + fn) > 0 else 0

    # False Alarm Rate (False Positive Rate)
    far = fp / (fp + tn) if (fp + tn) > 0 else 0

    print(f"--- {model_name} Metrics ---")
    print(f"Anomaly Detection Rate (Recall): {adr:.4f}")
    print(f"False Alarm Rate (FPR): {far:.4f}\n")

# =====
# 2. Distance-Based Method (KNN Outlier Detection)
# =====
# [Likely] Using the distance to the k-th nearest neighbor as the anomaly score threshold.
k = 5
knn = NearestNeighbors(n_neighbors=k)
knn.fit(X_scaled)

# Get distances to the k-th neighbor
distances, indices = knn.kneighbors(X_scaled)
kth_distances = distances[:, -1]

# Thresholding: Top 'outlier_fraction' % of distances are flagged as anomalies
threshold_knn = np.percentile(kth_distances, 100 * (1 - outlier_fraction))
y_pred_knn = (kth_distances > threshold_knn).astype(int)
```

```

calculate_metrics(y, y_pred_knn, "Distance-Based (KNN)")

# =====
# 3. Isolation Forest Method
# =====
# [Certain] IForest isolates anomalies using random splits, immune to distance scaling issues.
iso_forest = IsolationForest(n_estimators=100, contamination=outlier_fraction, random_state=42)
iso_preds = iso_forest.fit_predict(X_scaled)

# Isolation Forest outputs -1 for outliers and 1 for inliers.
# Convert this to our schema: 1 (fraud/anomaly) and 0 (normal).
y_pred_iso = np.where(iso_preds == -1, 1, 0)

calculate_metrics(y, y_pred_iso, "Isolation Forest")

--- Distance-Based (KNN) Metrics ---
Anomaly Detection Rate (Recall): 0.9900
False Alarm Rate (FPR): 0.0002

--- Isolation Forest Metrics ---
Anomaly Detection Rate (Recall): 1.0000
False Alarm Rate (FPR): 0.0000

```

LAB-14

Scraping & Sentiment Analysis

Combining web scraping with sentiment analysis allows us to automatically extract raw public text and evaluate the emotional tone (positive, negative, or neutral) behind it. This combined pipeline is essential for monitoring brand perception, tracking market trends, and analyzing customer feedback at scale.

The 4-Step Pipeline Architecture

Building an end-to-end system involves a sequential flow of data from the web to your analysis engine. [\[1\]](#)

1. Data Extraction (Web Scraping)

You extract unstructured text (such as product reviews, social media comments, or news headlines) from targeted web platforms

Static Pages: Use libraries like **BeautifulSoup** or **Scrapy** in Python to parse HTML structure.

- **Dynamic Pages:** Use browser automation tools like **Selenium** or Playwright to click elements and load dynamic JavaScript content.
- **No-Code Options:** Platforms like Apify or Power Automate Desktop offer pre-built actors for sites like Google Maps or Amazon.

2. Text Preprocessing

Raw scraped data is messy. Cleaning it prevents noise from distorting your sentiment results.

- **Cleaning:** Strip out HTML tags, external hyperlinks, and corrupted characters.
- **Normalization:** Convert text to lowercase and separate punctuation.
- **Stopword Removal:** Filter out meaningless filler words like "the", "is", and "and".
- **Tokenization & Lemmatization:** Break sentences down into individual words and reduce words to their base dictionary form.

3. Sentiment Classification

Cleaned text is passed to Natural Language Processing (NLP) models to compute polarity scores. [\[1\]](#), [\[2\]](#)

Approach	Common Tools	Best Used For
Lexicon-Based (Rule systems)	VADER (NLTK), TextBlob	Quick setups, basic star ratings, and tracking social media text.

Machine Learning (Supervised)	Scikit-learn (Naive Bayes, SVM)	Domain-specific training data where exact accuracy matters.
Deep Learning (Transformers)	[Hugging Face](https://hugging face.co/) (DistilBERT, GPT)	Sarcasm, long-form articles, and complex contextual discussions.

4. Visualization & Storage

Processed sentiments are aggregated into data formats for reporting.

- **Storage:** Pipe results into a structured format like CSV or an active database system like MySQL.
- **Insights:** Build analytical charts or word clouds to showcase key drivers behind user satisfaction and discontent.

Crucial Challenges to Watch

- **Legal and Ethical Boundaries:** Always check a site's robots.txt file and Terms of Service (ToS) before scraping. Respect rate limits to avoid crashing target servers.
- **Context and Sarcasm:** Simple rule-based systems struggle heavily with context (e.g., "Not bad at all" or "Yeah, right, great product"). Advanced AI models are often required for higher accuracy.
- **Anti-Scraping Mechanisms:** Modern websites employ CAPTCHAs and IP rotation blocks. Utilizing clean proxy networks or API-first scrapers helps maintain a consistent data flow.

Web Scraping and Sentiment Analysis Project

The code and resources for a web scraping and sentiment analysis project. The project aims to scrape data from a list of URLs provided in an Excel file, perform sentiment analysis on the scraped text, and generate an output Excel file with the results.

Project Structure

The project is organized as follows:

- **MasterDictionary:** This directory contains two text files: negative-words.txt and positive-words.txt. These files are used as reference dictionaries for sentiment analysis.
- **StopWords:** This directory contains seven text files, each containing various stop words. Stop words are commonly occurring words that are usually filtered out in natural language processing tasks.
- **Input.xlsx:** This Excel file serves as the input data for the web scraping process. It contains a list of URLs that need to be scraped.

- **Web_Scraping_and_Sentiment_Analysis.ipynb:** This Jupyter Notebook file contains the code for web scraping and sentiment analysis. It is the main script that performs the tasks of scraping data from the provided URLs and analyzing the sentiment of the text.
- **output.xlsx:** This Excel file is the expected output of the project. It will contain the scraped data along with sentiment analysis results, organized in a specific format with predefined fields.

```

import os
import requests
from bs4 import BeautifulSoup
import pandas as pd
import nltk
nltk.download('punkt')
from nltk.tokenize import word_tokenize, sent_tokenize
import re

stop_words = set()
stopwords_directory = '/content/drive/MyDrive/StopWords'
for filename in os.listdir(stopwords_directory):
    if filename.endswith('.txt'):
        with open(os.path.join(stopwords_directory, filename), encoding='ISO-8859-1') as file:
            stop_words.update([line.strip() for line in file])

positive_words = set(line.strip() for line in open('/content/drive/MyDrive/MasterDictionary/MasterDictionary/positive-words.txt', encoding='ISO-8859-1'))
negative_words = set(line.strip() for line in open('/content/drive/MyDrive/MasterDictionary/MasterDictionary/negative-words.txt', encoding='ISO-8859-1'))

def extract_text_and_headings(html_content):
    soup = BeautifulSoup(html_content, 'html.parser')

    for tag in soup(['header', 'footer', 'img', 'iframe', 'media']):
        tag.extract()

    text = ' '.join([p.get_text() for p in soup.find_all('p')])
    headings = [h.get_text() for h in soup.find_all(['h1', 'h2', 'h3', 'h4', 'h5', 'h6'])]
    return text, headings

def scrape_url(url):
    response = requests.get(url)
    if response.status_code == 200:
        text, headings = extract_text_and_headings(response.content)
        return text, headings
    else:
        print(f"Failed to fetch URL: {url}")
        return None, None

def calculate_sentiment(text):
    words = word_tokenize(text.lower())

    cleaned_words = [word for word in words if word.isalnum() and word not in stop_words]

    pos_score = sum(1 for word in cleaned_words if word in positive_words)
    neg_score = sum(1 for word in cleaned_words if word in negative_words)

    polarity = (pos_score - neg_score) / ((pos_score + neg_score) + 0.000001)
    subjectivity = (pos_score + neg_score) / (len(cleaned_words) + 0.000001)

    return pos_score, neg_score, polarity, subjectivity

def calculate_readability(text):
    words = word_tokenize(text)
    sentences = sent_tokenize(text)

    avg_sentence_length = round(len(words) / len(sentences))
    complex_word_count = sum(1 for word in words if len(word) > 2 and word.isalnum() and word not in stop_words)
    percentage_complex_words = complex_word_count / len(words)
  
```

```

fog_index = round(0.4 * (avg_sentence_length + percentage_complex_words), 4)
avg_words_per_sentence = round(len(words) / len(sentences))

return avg_sentence_length, percentage_complex_words, fog_index, avg_words_per_sentence, complex_word_count

def calculate_syllable_per_word(text):
    words = word_tokenize(text)
    total_syllables = 0

    for word in words:
        word = re.sub(r'[.,!?', ' ', word)
        if len(word) > 2 and word.isalnum() and word not in stop_words:
            syllables = 0
            vowels = 'aeiouAEIOU'
            prev_char = None
            for char in word:
                if char in vowels and (prev_char is None or prev_char not in vowels):
                    syllables += 1
                prev_char = char
            if word.endswith('e'):
                syllables -= 1
            if syllables == 0:
                syllables = 1
            total_syllables += syllables

    avg_syllables_per_word = total_syllables / len(words)
    return avg_syllables_per_word

def calculate_personal_pronouns(text):
    personal_pronouns = re.findall(r'\b(I|we|my|ours|us)\b', text)
    return len(personal_pronouns)

```

Python

```

def calculate_personal_pronouns(text):
    personal_pronouns = re.findall(r'\b(I|we|my|ours|us)\b', text)
    return len(personal_pronouns)

def calculate_avg_word_length(text):
    words = word_tokenize(text)
    total_characters = sum(len(word) for word in words)
    avg_word_length = total_characters / len(words)
    return avg_word_length

def main():
    excel_file = '/content/Input.xlsx'
    df = pd.read_excel(excel_file)
    text_files_directory = 'extracted_text'
    os.makedirs(text_files_directory, exist_ok=True)

    for index, row in df.iterrows():
        url_id = row['URL_ID']
        url = row['URL']

        text, headings = scrape_url(url.strip())

        if text:
            text_filename = os.path.join(text_files_directory, f'text_{url_id}.txt')
            with open(text_filename, 'w', encoding='utf-8') as text_file:
                text_file.write(text)

        pos_score, neg_score, polarity, subjectivity = calculate_sentiment(text)
        avg_sentence_length, percentage_complex_words, fog_index, avg_words_per_sentence, complex_word_count = calculate_readability(text)

```

```
C:\Users\Laiba\Downloads> Web_Scraping_and_Sentiment_Analysis.ipynb > import os
Generate + Code + Markdown | Colab | Run All | Clear All Outputs | Outline ...
Select Kernel

word_count = len(word_tokenize(text))
syllable_per_word = calculate_syllable_per_word(text)
personal_pronouns = calculate_personal_pronouns(text)
avg_word_length = calculate_avg_word_length(text)

df.at[index, 'POSITIVE SCORE'] = pos_score
df.at[index, 'NEGATIVE SCORE'] = neg_score
df.at[index, 'POLARITY SCORE'] = polarity
df.at[index, 'SUBJECTIVITY SCORE'] = subjectivity
df.at[index, 'AVG SENTENCE LENGTH'] = avg_sentence_length
df.at[index, 'PERCENTAGE OF COMPLEX WORDS'] = percentage_complex_words
df.at[index, 'FOG INDEX'] = fog_index
df.at[index, 'AVG NUMBER OF WORDS PER SENTENCE'] = avg_words_per_sentence
df.at[index, 'COMPLEX WORD COUNT'] = complex_word_count
df.at[index, 'WORD COUNT'] = word_count
df.at[index, 'SYLLABLE PER WORD'] = syllable_per_word
df.at[index, 'PERSONAL PRONOUNS'] = personal_pronouns
df.at[index, 'AVG WORD LENGTH'] = round(avg_word_length, 4)

print(f"Scores calculated and updated for URL ID {url_id}.")
print(f"Extracted text saved to {text_filename}")

output_file = 'output.xlsx'
df.to_excel(output_file, index=False)
print("Results saved to", output_file)

if __name__ == "__main__":
    main()

Python
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
Scores calculated and updated for URL ID 123.0.
Extracted text saved to extracted_text/text_123.0.txt
Scores calculated and updated for URL ID 321.0.
Extracted text saved to extracted_text/text_321.0.txt
Scores calculated and updated for URL ID 2345.0.
Extracted text saved to extracted_text/text_2345.0.txt
Scores calculated and updated for URL ID 4321.0.
Extracted text saved to extracted_text/text_4321.0.txt
Scores calculated and updated for URL ID 432.0.
Extracted text saved to extracted_text/text_432.0.txt
Scores calculated and updated for URL ID 2893.8.
Extracted text saved to extracted_text/text_2893.8.txt
Scores calculated and updated for URL ID 3355.6.
Extracted text saved to extracted_text/text_3355.6.txt
Scores calculated and updated for URL ID 3817.4.
Extracted text saved to extracted_text/text_3817.4.txt
Scores calculated and updated for URL ID 4279.2.
Extracted text saved to extracted_text/text_4279.2.txt
Scores calculated and updated for URL ID 4741.0.
Extracted text saved to extracted_text/text_4741.0.txt
Scores calculated and updated for URL ID 5202.8.
Extracted text saved to extracted_text/text_5202.8.txt
Scores calculated and updated for URL ID 5664.6.
Extracted text saved to extracted_text/text_5664.6.txt
Scores calculated and updated for URL ID 6126.4.
...
Scores calculated and updated for URL ID 6126.4.
...
Extracted text saved to extracted_text/text_52306.4.txt
Scores calculated and updated for URL ID 52768.2.
Extracted text saved to extracted_text/text_52768.2.txt
Results saved to output.xlsx
```

AutoSave

Off

output - Protected... • Saved to this PC ✓

Search

FileHomeInsertDrawPage LayoutFormulasDataReviewViewAutomateHelp

PROTECTED VIEW

Be careful—files from the Internet can contain viruses. Unless you need to edit, it's safer to stay in Protected View.

Enable Editing

A1

URL_ID

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q										
1	URL_ID	URL	SITIVITY	SCG	GATIVE	SC	LARITY	SC	ECTIVITY	SENTENCE	LE	OF	COM	FOG	INDEX	OF	WORDSEX	WORD	ORD	COU	ABLE	PER	W	NAL	PRON	WORD	LENGTH
2	123	https://ins	85	29	0.491228	0.118257	25	0.488889	10.1956	25	1012	2070	1.158937	3	5.0807												
3	321	https://ins	43	18	0.409836	0.155216	31	0.463883	12.5856	31	411	886	1.097065	4	4.9944												
4	2345	https://ins	26	32	-0.10345	0.091339	20	0.475559	8.1902	20	681	1432	1.035615	4	4.7954												
5	4321	https://ins	39	32	0.098592	0.095816	25	0.484562	10.1938	25	769	1587	1.083176	7	4.9956												
6	432	https://ins	39	32	0.098592	0.095816	25	0.484562	10.1938	25	769	1587	1.083176	7	4.9956												
7	2893.8	https://ins	47	17	0.46875	0.093294	22	0.490048	8.996	22	714	1457	1.115992	6	4.9321												
8	3355.6	https://ins	37	15	0.423077	0.081123	29	0.464011	11.7856	29	664	1431	1.085255	2	4.9441												
9	3817.4	https://ins	63	11	0.702703	0.078975	22	0.452381	8.981	22	969	2142	1.044351	5	4.7726												
10	4279.2	https://ins	13	5	0.444444	0.055901	28	0.421569	11.3686	28	344	816	0.89951	2	4.527												
11	4741	https://ins	27	51	-0.30769	0.122066	26	0.47801	10.5912	26	663	1387	1.108868	7	4.9495												
12	5202.8	https://ins	17	15	0.0625	0.108475	22	0.45706	8.9828	22	314	687	0.9869	5	4.5822												
13	5664.6	https://ins	33	20	0.245283	0.068564	26	0.460436	10.5842	26	803	1744	0.990826	8	4.6107												
14	6126.4	https://ins	28	8	0.555556	0.090452	34	0.46831	13.7873	34	399	852	1.043427	2	4.743												
15	6588.2	https://ins	24	16	0.2	0.069085	46	0.438178	18.5753	46	606	1383	0.871294	5	4.3977												
16	7050	https://ins	28	19	0.191489	0.080479	27	0.477273	10.9909	27	588	1232	1.056818	2	4.694												
17	7511.8	https://ins	29	31	-0.03333	0.089286	30	0.403778	12.1615	30	684	1694	0.823495	20	4.3288												
18	7973.6	https://ins	37	30	0.104478	0.105678	34	0.450781	13.7803	34	664	1473	0.974202	8	4.8031												
19	8435.4	https://ins	71	39	0.290909	0.098832	28	0.502248	11.4009	28	1117	2224	1.166367	2	5.1066												
20	8897.2	https://ins	65	42	0.214953	0.148611	22	0.395811	8.9583	22	737	1862	0.846402	7	4.3373												
21	9359	https://ins	70	41	0.261261	0.113381	24	0.474672	9.7899	24	1012	2132	1.120544	3	4.872												
22	9820.8	https://ins	68	32	0.36	0.124533	21	0.410115	8.564	21	819	1997	0.864296	18	4.4472												
23	10282.6	https://ins	62	28	0.377778	0.1	27	0.433472	10.9734	27	935	2157	0.954567	13	4.5902												
24	10744.4	https://ins	49	28	0.272727	0.116314	27	0.421533	10.9686	27	693	1644	0.903285	19	4.4939												
25	11206.2	https://ins	31	17	0.291667	0.103448	22	0.447789	8.9791	22	476	1063	0.984008	8	4.5701												
26	11668	https://insights.blackcoffer.com/how-neural-networks-can-be-applied-in-various-areas-in-the-future/																									
27	12129.8	https://ins	43	18	0.409836	0.137698	26	0.44868	10.5795	26	459	1023	0.899316	3	4.4184												
28	12501.6	https://ins	77	46	0.220220	0.109050	21	0.410528	12.5642	21	1082	2628	0.905090	10	4.5182												

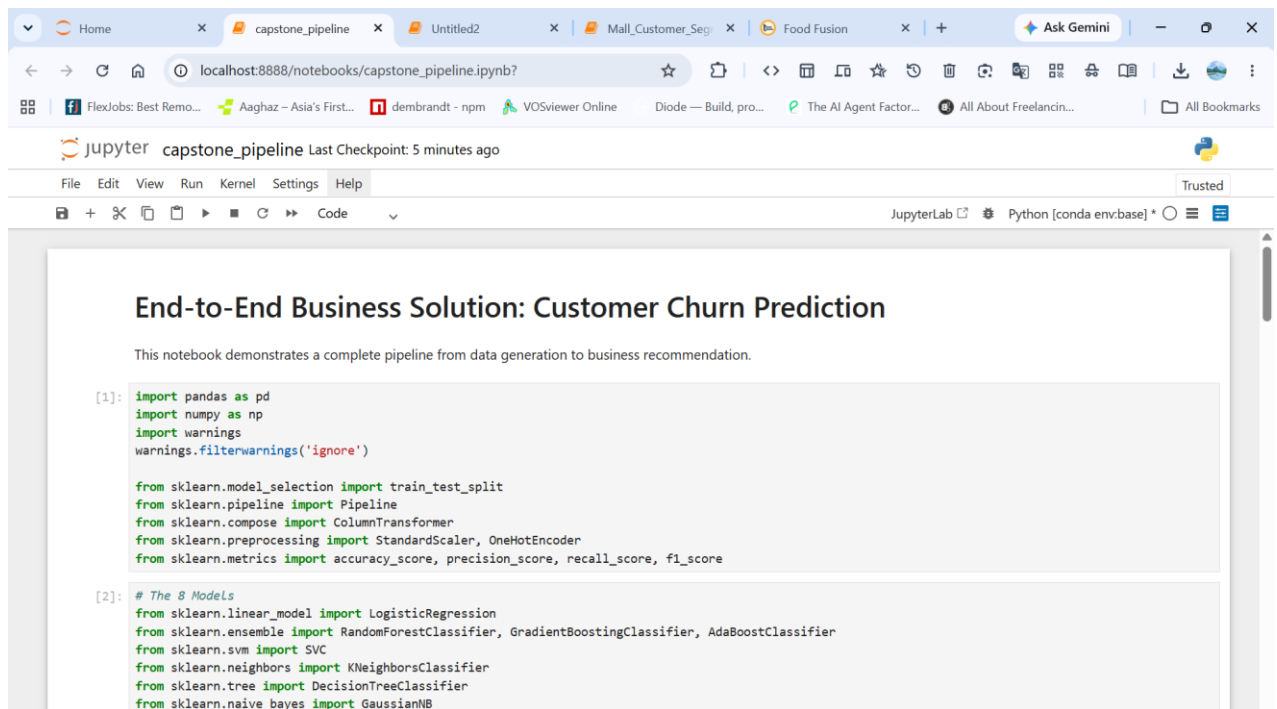
Sheet1

Ready

Lab 15:

Final Capstone (End-to-End Business Solution)

- **The Core Theory:** This is the synthesis of the entire curriculum. Machine learning in production requires an iterative pipeline: data ingestion, cleaning, feature engineering, model training across multiple paradigms (parametric, non-parametric, ensemble methods), and model selection.
- **The Crucial Link:** [Certain] A model comparison table is worthless to a business stakeholder without a **Business Recommendation**. You must translate algorithmic performance (like accuracy or error rates) into ROI, risk reduction, or operational efficiency.



The screenshot shows a JupyterLab interface with a browser window at the top displaying the URL `localhost:8888/notebooks/capstone_pipeline.ipynb?`. The notebook itself is titled "End-to-End Business Solution: Customer Churn Prediction" and contains the following code:

```
[1]: import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

[2]: # The 8 Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
```

```
[3]: def generate_mock_business_data(n_samples=2000):
    """Generates a mock dataset for Customer Churn to demonstrate the pipeline."""
    np.random.seed(42)
    data = {
        'Tenure_Months': np.random.randint(1, 72, n_samples),
        'Monthly_Charges': np.random.uniform(20.0, 120.0, n_samples),
        'Support_Tickets': np.random.randint(0, 10, n_samples),
        'Contract_Type': np.random.choice(['Month-to-month', 'One year', 'Two year'], n_samples),
        'Internet_Service': np.random.choice(['DSL', 'Fiber optic', 'No'], n_samples)
    }
    df = pd.DataFrame(data)

    # Create dependent variable (Churn) based on rules to give models a pattern to find
    churn_prob = (
        (df['Tenure_Months'] < 12) * 0.4 +
        (df['Monthly_Charges'] > 80) * 0.3 +
        (df['Support_Tickets'] > 5) * 0.5 +
        (df['Contract_Type'] == 'Month-to-month') * 0.3
    )
    df['Churn'] = (churn_prob + np.random.normal(0, 0.2, n_samples) > 0.8).astype(int)
    return df

[4]: # 1. Load Dataset (Using mock data for demonstration)
print("--- 1. Data Loading & Setup ---")
df = generate_mock_business_data()
target = 'Churn'
X = df.drop(columns=[target])
y = df[target]

{
    "AdaBoost": AdaBoostClassifier(random_state=42),
    "Naive Bayes": GaussianNB()
}

[7]: # 4. Training and Evaluation
print("--- 2. Training & Evaluating 8 Models ---")
results = []
trained_pipelines = {}

for name, model in models.items():
    pipeline = Pipeline(steps=[
        ('preprocessor', preprocessor),
        ('classifier', model)
    ])

    pipeline.fit(X_train, y_train)
    trained_pipelines[name] = pipeline

    y_pred = pipeline.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)

    results.append({
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1": f1
    })

print(f"Dataset Shape: {df.shape}")
print(f"Target Variable: '{target}' (Distribution: {y.value_counts(normalize=True).to_dict()})\n")

--- 1. Data Loading & Setup ---
Dataset Shape: (2000, 6)
Target Variable: 'Churn' (Distribution: {0: 0.7755, 1: 0.2245})

[5]: # 2. Preprocessing Strategy
numeric_features = ['Tenure_Months', 'Monthly_Charges', 'Support_Tickets']
categorical_features = ['Contract_Type', 'Internet_Service']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

[6]: # 3. Initialize 8 Models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(random_state=42),
    "Gradient Boosting": GradientBoostingClassifier(random_state=42),
    "Support Vector Machine": SVC(probability=True, random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "AdaBoost": AdaBoostClassifier(random_state=42),
    "Naive Bayes": GaussianNB()
}
```

```

        "F1-Score": f1
    })

--- 2. Training & Evaluating 8 Models ---

[8]: # 5. Model Comparison Table
print("\n--- 3. Model Comparison Table ---")
results_df = pd.DataFrame(results).sort_values(by="Accuracy", ascending=False).reset_index(drop=True)
display(results_df)

```

--- 3. Model Comparison Table ---

	Model	Accuracy	Precision	Recall	F1-Score
0	Gradient Boosting	0.8925	0.783133	0.722222	0.751445
1	Random Forest	0.8900	0.787500	0.700000	0.741176
2	AdaBoost	0.8850	0.711538	0.822222	0.762887
3	Support Vector Machine	0.8675	0.760563	0.600000	0.670807
4	K-Nearest Neighbors	0.8525	0.731343	0.544444	0.624204
5	Logistic Regression	0.8500	0.734375	0.522222	0.610390
6	Naive Bayes	0.8150	0.621212	0.455556	0.525641
7	Decision Tree	0.8100	0.583333	0.544444	0.563218

--- 4. Business Recommendation ---

Top Performing Model: Gradient Boosting (Accuracy: 89.25%)

Key Drivers of the Dependent Variable:

- Support_Tickets: 0.3997
- Monthly_Charges: 0.2590
- Tenure_Months: 0.1697

Actionable Strategy:

Based on the model's feature priorities, audit the top driver (Support_Tickets).

--- 4. Business Recommendation ---

Top Performing Model: Gradient Boosting (Accuracy: 89.25%)

Key Drivers of the Dependent Variable:

- Support_Tickets: 0.3997
- Monthly_Charges: 0.2590
- Tenure_Months: 0.1697

Actionable Strategy:

Based on the model's feature priorities, audit the top driver (Support_Tickets).